

CS 725 COURSE PROJECT • A BEGINNER'S GUIDE

Train on Less, Pay for Less?

Data Pruning with a Cheap Scorer Model

A step-by-step explanation of the ideas, the mathematics and the experiments behind the project, written for someone meeting these topics for the first time.



Each square is a training image; the amber ones are kept after pruning.

CS 725: Foundations of Machine Learning
Department of Computer Science and Engineering, IIT Bombay

Study notes for the project team • Autumn 2026

How to read this guide

This guide explains everything you need to understand, carry out and defend the project “*Data pruning with a cheap scorer model*”. It assumes only school-level mathematics: you should be comfortable with fractions, powers, the idea of a function, and basic derivatives. Anything beyond that (vectors, gradients, probability, softmax, and so on) is built up from scratch, and every formula that matters is *derived* rather than just stated.

The project in one paragraph. Training a neural network means showing it every training image many times, which is slow. Not all images are equally useful: some are easy and learned almost immediately, some are genuinely informative, and a few are confusing or even wrongly labelled. *Data pruning* throws away part of the training set before training, hoping to train faster without losing accuracy. To decide what to throw away, we give every image a *score*, and computing the scores needs a trained model. Our project asks whether a *small, cheap* model can do the scoring for a *bigger* model, how *early* in training the scores become reliable, and whether the whole process saves compute once we honestly count the cost of scoring.

How the chapters fit together.

- Chapters 1–4 build the foundations: what a model is, how it learns by gradient descent, and how image classifiers (CNNs, ResNets, softmax, cross-entropy) work.
- Chapters 5–8 are the heart of data pruning: which examples are easy or hard, how to score them (EL2N, GraNd, forgetting), and why the best pruning strategy depends on how much data you keep.
- Chapters 9–11 cover what is specific to our project: cheap scorers, honest compute accounting, and how all of this relates (and does not relate) to knowledge distillation.
- Chapters 12–14 are practical: how to run fair experiments, how to implement everything in PyTorch, and how to plan the work and prepare for the viva.
- Appendix A is a mathematics refresher. If a symbol in the main text is unfamiliar, look there first.

The boxes you will see.

Definition: a new term

A precise statement of what a word means.

In plain words

The same idea said informally, without symbols.

Worked example: numbers you can check

A small calculation done step by step. Every number in these boxes was checked by running it in Python.

Watch out

A common mistake or trap.

The figures in Chapters 7–10 come from small simulations run specifically for this guide (toy problems, not CIFAR). They show the *shape* of the effects we expect; the real CIFAR numbers are what your project will measure.

Contents

How to read this guide	i
1 The Big Picture	1
1.1 Training is expensive, and much of it is repetition	1
1.2 Data pruning in one picture	2
1.3 The catch that our project studies	2
1.4 What this project is not	2
1.5 What you will be able to do after this guide	3
2 Machine Learning Foundations	4
2.1 Data: inputs and labels	4
2.2 Models: functions with adjustable knobs	5
2.3 Loss: measuring how wrong we are	5
2.4 Training, validation and test data	5
2.5 Overfitting and underfitting	6
2.6 The learning curve of a single example	6
3 Learning by Gradient Descent	7
3.1 Derivatives: how much does the output change?	7
3.2 Gradients: derivatives for many knobs	7
3.3 Why step against the gradient? A derivation	8
3.4 The size of the gradient tells you how much you will learn	9
3.5 Stochastic gradient descent, mini-batches and epochs	9
3.6 Practical extras you will meet	10
4 Neural Networks for Images	11
4.1 A single neuron	11
4.2 Convolutions: neurons that look at small patches	11
4.3 Other building blocks	12

4.4	ResNets and skip connections	12
4.5	Softmax: turning logits into probabilities	13
4.6	Cross-entropy: the loss, derived from first principles	14
4.7	The most important derivation in this guide: $\nabla_z \ell = \mathbf{p} - \mathbf{y}$	14
4.8	Temperature (needed for Chapter 11)	15
5	The Datasets and the Life of a Training Example	17
5.1	CIFAR-10 and CIFAR-100	17
5.2	Four kinds of training example	17
5.3	Networks learn easy patterns first	18
5.4	Redundancy	18
5.5	Memorisation and the long tail	18
5.6	Class balance after pruning	19
6	Data Pruning: Definitions and Strategies	20
6.1	The formal setup	20
6.2	The three strategies	20
6.3	Why random pruning is a serious baseline	21
6.4	Static versus dynamic pruning	22
6.5	Two ways to train on less data: equal epochs or equal steps	22
7	Scoring Examples: EL2N, GraNd and Forgetting	23
7.1	The GraNd score: how hard does this example push?	23
7.2	The EL2N score: the length of the error vector	23
7.3	Why EL2N approximates GraNd: a derivation	24
7.4	How EL2N relates to confidence: a two-sided bound	25
7.5	Averaging over several scorer runs	26
7.6	The forgetting score	26
7.7	Comparing the scores	27
8	Why the Best Strategy Depends on How Much You Keep	28
8.1	A toy problem you can solve by hand	28
8.2	Random data: error shrinks like $1/n$	29
8.3	Keeping hard examples with a perfect scorer	29
8.4	Keeping hard examples with an <i>imperfect</i> scorer	30
8.5	Beyond one dimension: when keeping easy examples wins	31

8.6	The imperfect scorer in the teacher–student model	32
9	Cheap Scorers: Can a Small Model Choose for a Big One?	34
9.1	The idea of a proxy	34
9.2	Reasons to expect a cheap scorer to work	34
9.3	Reasons to expect it to fail	35
9.4	Measuring agreement between two scorers	35
9.4.1	Spearman rank correlation	35
9.4.2	Kendall’s tau	36
9.4.3	Overlap of the kept sets	36
9.5	The experiments that answer the question	37
10	Honest Compute Accounting	38
10.1	What to measure	38
10.2	The cost model	38
10.3	Worked examples	39
10.4	The baseline that is easy to forget	40
10.5	Amortisation: when the pruned set is reused	40
10.6	Equal steps: why it saves nothing	40
10.7	The cost table we will produce	41
11	How This Relates to Knowledge Distillation	42
11.1	What knowledge distillation is	42
11.2	The distillation loss	42
11.3	The gradient of the distillation term	43
11.4	Why the loss is multiplied by T^2	43
11.5	Dataset distillation: a different idea with a similar name	44
11.6	Side-by-side comparison	44
11.7	An optional bonus experiment that uses distillation	44
12	Doing Experiments Properly	45
12.1	Where the randomness comes from	45
12.2	Mean, standard deviation and the $n - 1$	45
12.3	Standard error and confidence intervals	46
12.4	Is the difference between two methods real?	46
12.5	A fair-comparison checklist	47

12.6 How many runs can we afford?	47
13 Implementation in PyTorch	49
13.1 File layout	49
13.2 Data	49
13.3 Models	50
13.4 Training	51
13.5 Scores	52
13.6 Pruning	53
13.7 The experiment grid	54
13.8 Analysis	55
13.9 Working on Kaggle or Colab	56
14 Project Plan and Viva Preparation	57
14.1 Timeline	57
14.2 Team roles	58
14.3 Likely viva questions, with short answers	58
14.4 A final checklist	59
A Mathematics Refresher	61
A.1 Vectors, dot products and norms	61
A.2 Matrices	61
A.3 Exponentials and logarithms	62
A.4 Derivative rules	62
A.5 Probability and expectation	62
A.6 Two sums	62
B Glossary	64
References	66

Chapter 1

The Big Picture

This chapter explains, without any mathematics, what the project is about and why it is worth doing. Everything here is made precise in later chapters.

1.1 Training is expensive, and much of it is repetition

A modern image classifier learns from a *training set*: a collection of example images, each with the correct answer (its *label*) attached. For our project the training set is CIFAR-10: 50,000 small colour photos, each labelled as one of ten classes (airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck).

The network does not learn from one look. It goes through all 50,000 images, adjusts itself a little after each small batch, and then does it all again: typically tens or hundreds of times. One complete pass over the training set is called an *epoch*. If a training run takes 100 epochs, the network processes $100 \times 50,000 = 5$ million image views.

Now think about how you would study for an exam from a book of 50,000 practice questions. After a few rounds you would notice that:

- many questions are **easy**: you got them right the first time and every time since, so re-reading them teaches you nothing new;
- some questions are **instructive**: they sit right at the edge of what you understand, and working through them improves you;
- a few questions are **broken**: the answer key is wrong, or the question is so ambiguous that no sensible answer exists. Studying these can actually make you worse.

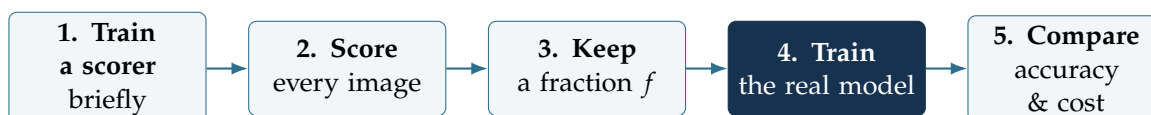
A sensible student would skip most of the easy questions and all the broken ones and spend their time on the instructive middle. **Data pruning** is the machine-learning version of this idea.

1.2 Data pruning in one picture

Definition: data pruning

Data pruning means choosing a subset of the training set before (or early in) training, and then training the model only on that subset. The fraction of data we keep is called the *keep fraction* f (for example $f = 0.5$ means we keep half).

To decide which examples to keep, we need a number for every training example that says how “hard” or “useful” it is. This number is its *score*. The process looks like this:



If everything works, step 4 is cheaper than training on all the data (because there is less data) and the final accuracy is almost as good.

1.3 The catch that our project studies

Look again at step 1. To know which images are easy or hard, we have to train a model first, at least for a while. **Scoring is not free.** If the scorer costs as much as the time we save in step 4, pruning has achieved nothing.

Most published work on data pruning reports a graph of “accuracy versus fraction of data kept” and treats the scoring step as already done. That is a reasonable way to study *which* examples matter, but it does not answer the practical question: *does pruning save me time overall?*

Our project attacks this directly with three questions.

1. **Cheap scorer.** Can a *small* network (fast to train) produce scores that are good enough to prune the data for a *bigger* network? (Chapter 9)
2. **Early scoring.** How early can we compute the scores: after 1 epoch, 5 epochs, 20 epochs? **Earlier means cheaper.** (Chapters 7 and 9)
3. **Honest accounting.** If we add the scoring cost to the training cost, do we actually come out ahead of just training on everything? (Chapter 10)

The central tension of the project. The more data we throw away, the more time we can save, but aggressive pruning is also where a *bad* score does the most damage. A cheap scorer is attractive exactly because it is cheap, but it is also more likely to be inaccurate. Chapter 8 shows this tension with a small example you can solve by hand.

1.4 What this project is not

Two nearby ideas are easy to confuse with ours.

- **Knowledge distillation** trains a small “student” model to copy the *soft predictions* of a big “teacher” model. There, information flows from big to small, and what is passed on is a probability for every class on every image. In our project, information flows from *small to big*, and what is passed on is only a *list of which images to keep*. Chapter 11 explains distillation properly and compares the two.
- **Dataset distillation** *creates* a small set of artificial images that summarise the dataset. We never create images; we only *select* real ones.

1.5 What you will be able to do after this guide

By the end you should be able to:

- explain softmax, cross-entropy and gradient descent, and derive the key formula $\nabla_z \ell = p - y$;
- define and compute EL2N, GraNd and forgetting scores, and prove how they relate;
- explain, with a worked example, why keeping hard examples is best when you keep a lot of data but risky when you keep very little;
- compare two rankings with Spearman and Kendall correlations;
- write down and use a cost model to decide whether pruning saves compute;
- run experiments with multiple seeds and judge whether a difference is real;
- implement the whole pipeline in PyTorch on a free GPU.

Chapter 2

Machine Learning Foundations

Before we can talk about pruning data, we need a clear picture of what “learning from data” means. This chapter introduces the vocabulary used in the rest of the guide.

2.1 Data: inputs and labels

Definition: training set

A *training set* is a list of N pairs

$$\mathcal{D} = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}.$$

Each x_i is an *input* (for us, an image) and each y_i is its *label* (the correct class). The index i just numbers the examples.

How is an image a list of numbers? A CIFAR image is 32×32 pixels, and each pixel has three colour intensities (red, green, blue) between 0 and 255. So one image is $32 \times 32 \times 3 = 3072$ numbers. We write $x \in \mathbb{R}^{3072}$, which reads “ x is a list (vector) of 3072 real numbers”. In practice we divide by 255 and then shift and scale each colour channel so the numbers are roughly between -2 and 2 ; this *normalisation* just makes training better behaved.

How is a class a number? We number the classes $0, 1, \dots, K-1$, where K is the number of classes ($K = 10$ for CIFAR-10, $K = 100$ for CIFAR-100). So $y_i \in \{0, \dots, K-1\}$. Later it will be convenient to write the label as a *one-hot vector*: a vector of length K that is all zeros except for a 1 in position y .

Worked example: one-hot encoding

With classes (cat, dog, car) numbered $(0, 1, 2)$, the label “dog” is $y = 1$ and its one-hot vector is $\mathbf{y} = (0, 1, 0)$.

2.2 Models: functions with adjustable knobs

Definition: model and parameters

A *model* is a function h_w that takes an input x and produces a prediction. The vector w contains the model's *parameters* (also called *weights*): the adjustable numbers that learning will tune.

A tiny example: to predict house price from area, the model $h_w(x) = w_1x + w_0$ has two parameters, a slope w_1 and an intercept w_0 . A ResNet-18 is the same idea on a vastly larger scale: about 11 million parameters.

For classification, the network outputs K numbers called *logits*, one per class, and a function called *softmax* turns them into probabilities (Chapter 4). The predicted class is the one with the highest probability.

2.3 Loss: measuring how wrong we are

To improve a model we need a number that says how bad its current predictions are.

Definition: loss

A *loss function* $\ell(h_w(x), y) \geq 0$ measures how wrong the prediction on one example is; it is 0 (or close to it) when the prediction is perfect. The *training loss* is the average over the training set:

$$L(w) = \frac{1}{N} \sum_{i=1}^N \ell(h_w(x_i), y_i).$$

For classification we use the *cross-entropy* loss, derived in Chapter 4. For now just remember: **small loss = confident and correct, large loss = wrong or unsure.**

In plain words

Learning = searching for the knob settings w that make the average loss on the training set small.

2.4 Training, validation and test data

A model that scores perfectly on its training set may simply have memorised it. What we really care about is performance on *new* images.

Definition: generalisation and the test set

The *test set* is a separate collection of labelled examples that is never used for training or for any decision during the project. *Test accuracy* (the fraction of test images classified correctly) measures how well the model *generalises*. CIFAR-10 provides 10,000 test images.

Never tune on the test set

If you try many settings and pick the one with the best test accuracy, the test set is no longer “new” data and your reported number is optimistic. For choosing settings (learning rate, number of epochs), hold out part of the training set as a *validation set*. In our project the settings are mostly fixed in advance, so we report test accuracy at the end of each run; we just must not choose between methods by peeking at test results run after run.

2.5 Overfitting and underfitting

- **Underfitting:** the model is too simple or has not trained long enough; training accuracy and test accuracy are both low.
- **Overfitting:** the model has fitted details of the training set that do not carry over, such as noise or wrong labels; training accuracy is high but test accuracy is noticeably lower.

Large networks can reach 100% training accuracy on CIFAR-10, even if you randomly shuffle the labels [8]. This means the network *can* memorise mislabelled images. It is one reason why removing broken examples might help, and why “hard” examples need careful thought: a hard example might be informative, or it might just be wrong.

2.6 The learning curve of a single example

Here is the observation that makes data pruning possible. During training, we can watch each individual training example:

- Is it classified correctly at the end of epoch 1? Epoch 2? ...
- What is its loss at each epoch?

Different examples behave very differently. Easy examples are correct after one or two epochs and stay correct. Hard examples are learned late, or flip between correct and incorrect. Mislabelled examples are typically learned last, only when the network starts memorising. Chapter 5 looks at this in detail, and Chapter 7 turns these behaviours into scores.

Check yourself

1. How many numbers describe one CIFAR image? (*Answer: 3072.*)
2. Write the one-hot vector for class 7 when $K = 10$. (*Answer: (0, 0, 0, 0, 0, 0, 0, 1, 0, 0).*)
3. Why do we not use test accuracy to choose between hyperparameter settings? (*Answer: the reported test accuracy would then be optimistically biased.*)

Chapter 3

Learning by Gradient Descent

We now know that learning means making the training loss small. This chapter explains *how* a network does that. The idea is gradient descent, and it is the reason the “gradient” appears in the scores of Chapter 7.

3.1 Derivatives: how much does the output change?

For a function of one variable $f(w)$, the derivative $f'(w)$ tells us the slope: if we nudge w by a small amount Δ , the output changes by about $f'(w) \Delta$:

$$f(w + \Delta) \approx f(w) + f'(w) \Delta. \quad (3.1)$$

This is the *first-order Taylor approximation*. It is exact for straight lines and very accurate for any smooth function when Δ is small.

Worked example: a derivative as a slope

Let $f(w) = (w - 3)^2$. Then $f'(w) = 2(w - 3)$. At $w = 0$ the slope is -6 : increasing w slightly *decreases* f , by about 6 times the step. Indeed $f(0) = 9$ and $f(0.1) = 8.41$, a decrease of $0.59 \approx 6 \times 0.1$.

Chain rule. If $f(w) = g(u(w))$, then $f'(w) = g'(u(w)) \cdot u'(w)$. Neural networks are long chains of simple functions, so the chain rule is the tool that lets us differentiate them. The algorithm that applies it efficiently, layer by layer from the output back to the input, is called *backpropagation*.

3.2 Gradients: derivatives for many knobs

A network has millions of parameters $\mathbf{w} = (w_1, \dots, w_d)$. The *partial derivative* $\partial L / \partial w_j$ is the slope of L when we change only w_j and hold the rest fixed.

Definition: gradient

The *gradient* of L is the vector of all partial derivatives:

$$\nabla L(\mathbf{w}) = \left(\frac{\partial L}{\partial w_1}, \frac{\partial L}{\partial w_2}, \dots, \frac{\partial L}{\partial w_d} \right).$$

The many-variable version of (3.1) is: if we change $\mathbf{w}$ by a small vector Δ ,

$$L(\mathbf{w} + \Delta) \approx L(\mathbf{w}) + \nabla L(\mathbf{w}) \cdot \Delta, \quad (3.2)$$

where $\mathbf{a} \cdot \mathbf{b} = \sum_j a_j b_j$ is the dot product (Appendix A).

3.3 Why step against the gradient? A derivation

We want to choose a small step Δ that decreases L as much as possible.

Proposition 3.1. *Among all steps of a fixed small length, the one that decreases L fastest points in the direction of $-\nabla L(\mathbf{w})$.*

Proof. By (3.2), the change in loss is approximately $\nabla L \cdot \Delta$. By the Cauchy–Schwarz inequality (Appendix A),

$$\nabla L \cdot \Delta \geq -\|\nabla L\| \|\Delta\|,$$

with equality exactly when Δ points opposite to ∇L . So for a fixed length $\|\Delta\|$, the most negative change (the biggest decrease) is achieved by $\Delta = -\eta \nabla L$ for some $\eta > 0$. $\square$

Plugging $\Delta = -\eta \nabla L$ into (3.2):

$$L(\mathbf{w} - \eta \nabla L) \approx L(\mathbf{w}) - \eta \|\nabla L(\mathbf{w})\|^2. \quad (3.3)$$

Since $\|\nabla L\|^2 \geq 0$, the loss goes down (as long as η is small enough for the approximation to hold). This gives the rule:

Definition: gradient descent

Starting from some $\mathbf{w}_0$, repeat

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \nabla L(\mathbf{w}_t).$$

The positive number η is the *learning rate* (step size).

In plain words

Stand on a hillside in fog. You cannot see the valley, but you can feel which way the ground slopes. Take a small step downhill. Repeat. The gradient is “which way is uphill, and how steep”; we step the opposite way.

Worked example: gradient descent by hand

Minimise $f(w) = (w - 3)^2$, whose minimum is at $w = 3$. Use $\eta = 0.1$ and start at $w_0 = 0$. The update is $w_{t+1} = w_t - 0.1 \cdot 2(w_t - 3)$.

t	0	1	2	3	4	5	6
w_t	0	0.6	1.08	1.464	1.771	2.017	2.214

Each step closes 20% of the remaining gap to 3, because $w_{t+1} - 3 = 0.8(w_t - 3)$. With $\eta = 1.1$ instead, the factor would be $1 - 2.2 = -1.2$ and the iterates would jump back and forth ever further: a learning rate that is too large makes training diverge.

3.4 The size of the gradient tells you how much you will learn

Equation (3.3) has a lesson that returns in Chapter 7: **the decrease in loss from one step is proportional to $\|\nabla L\|^2$** . If an example produces a tiny gradient, training on it barely changes anything. If it produces a large gradient, it pushes the network a lot. This is the reasoning behind the GraNd score: *an example whose gradient is small is contributing little to learning*.

3.5 Stochastic gradient descent, mini-batches and epochs

The training loss is an average over all N examples, and so is its gradient:

$$\nabla L(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N \nabla \ell_i(\mathbf{w}), \quad \text{where } \ell_i(\mathbf{w}) = \ell(h_{\mathbf{w}}(\mathbf{x}_i), y_i).$$

Computing this over all 50,000 images for every single step is too slow. Instead we estimate it from a small random *mini-batch* B of, say, 128 examples:

$$\mathbf{g}_B = \frac{1}{|B|} \sum_{i \in B} \nabla \ell_i(\mathbf{w}).$$

Proposition 3.2. *If the mini-batch is drawn uniformly at random, $\mathbf{g}_B$ is an unbiased estimate of the full gradient: $\mathbb{E}[\mathbf{g}_B] = \nabla L(\mathbf{w})$.*

Proof. Take a single example drawn uniformly at random, with index I . Each index has probability $1/N$, so $\mathbb{E}[\nabla \ell_I] = \sum_{i=1}^N \frac{1}{N} \nabla \ell_i = \nabla L$. A mini-batch average is an average of $|B|$ such draws, and the expectation of an average is the average of the expectations (linearity of expectation, Appendix A), so it also equals ∇L . $\square$

This algorithm is *stochastic gradient descent* (SGD). In practice we shuffle the training set, cut it into consecutive mini-batches, and take one step per mini-batch. Going through all of them once is one *epoch*; with $N = 50,000$ and batch size 128, one epoch is $\lceil 50000/128 \rceil = 391$ steps.

Cost of training. The time for a training run is roughly proportional to

$$(\text{number of examples processed}) = (\text{epochs}) \times (\text{training-set size}).$$

If we keep a fraction f of the data and train for the same number of epochs, the run costs about f times as much. This simple fact is the basis of the compute accounting in Chapter 10.

3.6 Practical extras you will meet

- **Momentum:** instead of stepping along the current mini-batch gradient only, keep a running average of recent gradients and step along that. It smooths out noise. The standard CIFAR recipe uses SGD with momentum 0.9.
- **Weight decay:** adds a small penalty $\frac{\lambda}{2} \|w\|^2$ to the loss to keep weights small (typical $\lambda = 5 \times 10^{-4}$).
- **Learning-rate schedule:** start with a larger η (e.g. 0.1) and decrease it during training, for example with a cosine curve down to 0. Large steps early make fast progress; small steps late settle into a good minimum.
- **Data augmentation:** at every epoch, randomly crop (with padding) and horizontally flip each image. The network never sees exactly the same picture twice, which reduces overfitting.

Check yourself

1. In the worked example, what fraction of the gap remains after 10 steps? (*Answer:* $0.8^{10} \approx 0.107$.)
2. With $N = 25,000$ (half of CIFAR-10) and batch size 128, how many steps is one epoch? (*Answer:* $\lceil 25000/128 \rceil = 196$.)
3. Why does halving the data (same epochs) roughly halve the training time? (*Answer:* *time is proportional to the number of examples processed.*)

Chapter 4

Neural Networks for Images

This chapter explains the models we will train (small CNNs and ResNet-18) and, most importantly, derives the two formulas that every pruning score is built on: the *softmax* and the gradient of the *cross-entropy* loss.

4.1 A single neuron

A neuron takes inputs $x_1, \dots, x_n$, multiplies each by a weight, adds a bias, and passes the result through a simple non-linear function:

$$a = \phi \left(\sum_{j=1}^n w_j x_j + b \right).$$

The most common choice is the **ReLU**, $\phi(u) = \max(0, u)$: it passes positive values unchanged and replaces negative values by 0. Without a non-linearity, stacking layers would be pointless, because a chain of linear functions is itself just one linear function.

A *layer* is many neurons side by side, reading the same inputs. A *deep network* is many layers in sequence, each reading the previous layer's outputs.

4.2 Convolutions: neurons that look at small patches

For images, a fully connected layer would be wasteful: a neuron connected to all 3072 inputs ignores the fact that nearby pixels belong together. A *convolutional layer* instead uses small filters (typically 3×3) that slide across the image. Each position computes a weighted sum of the 3×3 patch underneath.

Worked example: a filter that finds vertical edges

Take one colour channel of a 3×3 patch that is bright on the left and dark on the right, and a filter that subtracts the right column from the left column:

$$\text{patch} = \begin{pmatrix} 9 & 5 & 1 \\ 8 & 4 & 0 \\ 9 & 6 & 2 \end{pmatrix}, \quad \text{filter} = \begin{pmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{pmatrix}.$$

Multiply matching entries and add them: $(9 - 1) + (8 - 0) + (9 - 2) = 23$, a large response. On a uniform patch (all entries equal) the response is 0. Sliding the filter over the whole image produces a map that “lights up” where there are vertical edges.

A real layer has many filters (for example 64), each producing its own map (a *channel*), and each filter looks at all input channels at once. The network *learns* the filter values by gradient descent. Early layers learn edges and colours; deeper layers combine them into textures, parts and eventually whole objects.

Counting the work in a convolution. A layer with a $k \times k$ filter, C_{in} input channels, C_{out} output channels and an $H \times W$ output map performs

$$\text{multiply-adds} = k^2 \cdot C_{\text{in}} \cdot C_{\text{out}} \cdot H \cdot W, \quad (4.1)$$

because each of the $C_{\text{out}}HW$ outputs is a weighted sum of $k^2 C_{\text{in}}$ inputs. For $k = 3$, $C_{\text{in}} = C_{\text{out}} = 64$ and a 32×32 output, this is $3^2 \cdot 64 \cdot 64 \cdot 32 \cdot 32 = 37,748,736$ multiply-adds *for one image and one layer*. This is why big networks are slow and small ones are fast, which is the whole reason a cheap scorer is cheap (Chapter 10).

4.3 Other building blocks

- **Pooling / striding** shrinks the spatial size (e.g. $32 \times 32 \rightarrow 16 \times 16$) so later layers see a larger area of the image and do less work per channel.
- **Batch normalisation** rescales each channel’s outputs to have roughly zero mean and unit variance over the mini-batch. It makes training faster and more stable.
- **Global average pooling** at the end averages each channel over all positions, giving one number per channel.
- **A final linear layer** maps these numbers to K logits $\mathbf{z} = (z_1, \dots, z_K)$, one per class.

4.4 ResNets and skip connections

Very deep “plain” networks are hard to train. The reason is visible through the chain rule: the gradient reaching an early layer is a *product* of many layer-by-layer derivatives. If these are typically smaller than 1, the product shrinks towards 0 (*vanishing gradients*) and early layers stop learning.

A *residual block* [10] adds a shortcut:

$$\mathbf{y} = \mathbf{x} + F(\mathbf{x}),$$

where F is a couple of convolution layers. By the chain rule (written for scalars to keep it simple),

$$\frac{\partial \ell}{\partial \mathbf{x}} = \frac{\partial \ell}{\partial \mathbf{y}} (1 + F'(\mathbf{x})).$$

The “1+” is the key: even if $F'(\mathbf{x})$ is small, the gradient passes through the block almost unchanged.

Worked example: why the shortcut matters

Stack 20 layers. In a plain network where each layer's derivative is 0.5, the gradient at the bottom is multiplied by $0.5^{20} \approx 9.5 \times 10^{-7}$: essentially zero. In a residual network where each block's F' is -0.05 , the factor per block is 0.95 and the product is $0.95^{20} \approx 0.36$: a healthy signal.

ResNet-18 for CIFAR has one 3×3 convolution at the start, then four stages of two residual blocks each (64, 128, 256 and 512 channels; each block has two convolutions), then global average pooling and a linear layer. That is $1 + 4 \times 2 \times 2 + 1 = 18$ weight layers, and about 11 million parameters. It reaches roughly 94–95% test accuracy on CIFAR-10 with a standard recipe.

A “tiny CNN” scorer could be, for example, three convolution layers with 32, 64 and 128 channels, each followed by ReLU and pooling, then a linear layer. It has far fewer multiply–adds than ResNet-18 and trains in a small fraction of the time, while still reaching a respectable accuracy (typically somewhere around 70–85% on CIFAR-10, depending on the design and training length). Whether its opinion of which images are hard agrees with ResNet-18's is exactly the question of Chapter 9.

4.5 Softmax: turning logits into probabilities

The logits $z_1, \dots, z_K$ can be any real numbers. To read them as probabilities we need numbers that are positive and add up to 1.

Definition: softmax

$$p_k = \text{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}, \quad k = 1, \dots, K.$$

Each e^{z_k} is positive, and dividing by their sum makes them add up to 1. Larger logits get larger probabilities, and because of the exponential, a logit that is ahead by a couple of units dominates.

Worked example: computing a softmax

Classes (cat, dog, car), logits $\mathbf{z} = (2.0, 1.0, 0.1)$.

$$e^{2.0} = 7.389, \quad e^{1.0} = 2.718, \quad e^{0.1} = 1.105, \quad \text{sum} = 11.212,$$

$$\mathbf{p} = (7.389, 2.718, 1.105) / 11.212 = (0.659, 0.242, 0.099).$$

The model says “cat” with probability 0.659.

Proposition 4.1 (Softmax ignores a common shift). *For any constant c , $\text{softmax}(\mathbf{z} + c) = \text{softmax}(\mathbf{z})$, where $\mathbf{z} + c$ means adding c to every logit.*

Proof.
$$\frac{e^{z_k+c}}{\sum_j e^{z_j+c}} = \frac{e^c e^{z_k}}{e^c \sum_j e^{z_j}} = \frac{e^{z_k}}{\sum_j e^{z_j}}.$$

□

Software uses this to avoid overflow: it subtracts $\max_j z_j$ from all logits before exponentiating.

4.6 Cross-entropy: the loss, derived from first principles

How should we measure the quality of predicted probabilities? A natural principle is *maximum likelihood*: choose parameters that make the observed labels as probable as possible.

The model assigns probability $p_{y_i}(x_i)$ to the correct label of example i . Treating examples as independent, the probability of *all* the observed labels is the product

$$\prod_{i=1}^N p_{y_i}(x_i).$$

We want to make this large. Products of many small numbers are awkward, so take the logarithm (which is increasing, so it does not change where the maximum is) and flip the sign to get something to *minimise*:

$$-\log \prod_{i=1}^N p_{y_i}(x_i) = \sum_{i=1}^N (-\log p_{y_i}(x_i)).$$

Dividing by N gives the average cross-entropy loss.

Definition: cross-entropy loss

For one example with true class y and predicted probabilities p ,

$$\ell = -\log p_y = -\sum_{k=1}^K y_k \log p_k,$$

where the second form uses the one-hot vector y (only the $k = y$ term survives).

Worked example: loss values

With $p = (0.659, 0.242, 0.099)$ from before: if the true class is *cat*, $\ell = -\ln 0.659 = 0.417$; if the true class is *dog*, $\ell = -\ln 0.242 = 1.417$. **Being confident and wrong** costs a lot; as $p_y \rightarrow 0$ the loss grows without bound.

4.7 The most important derivation in this guide: $\nabla_z \ell = p - y$

To train with gradient descent we need the derivative of the loss with respect to the logits. We get it in two steps.

Lemma 4.2 (Derivative of softmax). *For all i, j ,*

$$\frac{\partial p_i}{\partial z_j} = p_i (\delta_{ij} - p_j),$$

where $\delta_{ij} = 1$ if $i = j$ and 0 otherwise.

Proof. Write $S = \sum_k e^{z_k}$, so $p_i = e^{z_i} / S$ and $\partial S / \partial z_j = e^{z_j}$.

Case $i = j$. By the quotient rule,

$$\frac{\partial p_i}{\partial z_i} = \frac{e^{z_i} S - e^{z_i} e^{z_i}}{S^2} = \frac{e^{z_i}}{S} - \left(\frac{e^{z_i}}{S} \right)^2 = p_i - p_i^2 = p_i(1 - p_i).$$

Case $i \neq j$. The numerator e^{z_i} does not depend on z_j , so

$$\frac{\partial p_i}{\partial z_j} = -\frac{e^{z_i} e^{z_j}}{S^2} = -p_i p_j.$$

Both cases are captured by $p_i(\delta_{ij} - p_j)$. □

Theorem 4.3 (Gradient of cross-entropy with respect to the logits). For $\ell = -\log p_y$ with $\mathbf{p} = \text{softmax}(\mathbf{z})$,

$$\frac{\partial \ell}{\partial z_j} = p_j - y_j \quad \text{for every } j, \quad \text{that is,} \quad \nabla_{\mathbf{z}} \ell = \mathbf{p} - \mathbf{y}.$$

Proof. By the chain rule and Lemma 4.2,

$$\frac{\partial \ell}{\partial z_j} = -\frac{1}{p_y} \frac{\partial p_y}{\partial z_j} = -\frac{1}{p_y} p_y (\delta_{yj} - p_j) = p_j - \delta_{yj}.$$

Finally, δ_{yj} is exactly the j -th entry of the one-hot vector $\mathbf{y}$. □

Read the formula $\mathbf{p} - \mathbf{y}$ in words: “what the model predicted minus what it should have predicted”. It is the *error vector* at the output. When the model is confident and correct, $\mathbf{p} \approx \mathbf{y}$ and the gradient is almost zero, so the example teaches almost nothing. When the model is wrong, the vector is large. The EL2N score of Chapter 7 is simply the length of this vector.

Worked example: the error vector

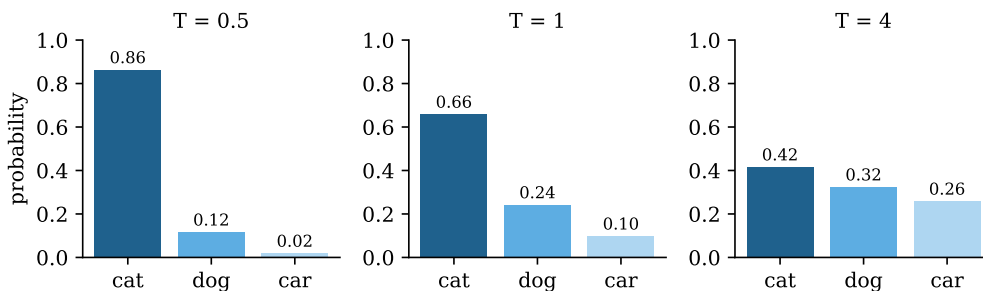
With $\mathbf{p} = (0.659, 0.242, 0.099)$:

- true class *cat*: $\mathbf{p} - \mathbf{y} = (-0.341, 0.242, 0.099)$. Gradient descent *raises* the cat logit and lowers the others.
- true class *dog*: $\mathbf{p} - \mathbf{y} = (0.659, -0.758, 0.099)$, a much larger push.

Notice that the entries of $\mathbf{p} - \mathbf{y}$ always add up to 0, because $\sum_j p_j = \sum_j y_j = 1$.

4.8 Temperature (needed for Chapter 11)

Sometimes we divide logits by a *temperature* $T > 0$ before the softmax: $\mathbf{p}^{(T)} = \text{softmax}(\mathbf{z}/T)$. A large T flattens the distribution and a small T sharpens it. For $\mathbf{z} = (2, 1, 0.1)$:



At $T = 4$ the “dark knowledge” becomes visible: the model thinks this cat looks more like a dog than a car. Knowledge distillation exploits this.

Check yourself

1. Show that the entries of $\mathbf{p} - \mathbf{y}$ sum to 0. (*Hint: both $\mathbf{p}$ and $\mathbf{y}$ sum to 1.*)
2. Compute $\text{softmax}(3, 2, 1.1)$ without a calculator, using a previous result. (*Answer: it equals $\text{softmax}(2, 1, 0.1) = (0.659, 0.242, 0.099)$ by shift invariance.*)
3. A 3×3 convolution maps 128 channels to 128 channels at 16×16 . How many multiply-adds per image? (*Answer: $9 \cdot 128 \cdot 128 \cdot 256 = 37,748,736$, the same as the 64-channel 32×32 layer.*)

Chapter 5

The Datasets and the Life of a Training Example

Data pruning is about treating training examples differently. So we first need to understand how different they are, and what happens to each one during training.

5.1 CIFAR-10 and CIFAR-100

Both datasets were introduced by Krizhevsky [11].

	CIFAR-10	CIFAR-100
Image size	32×32 colour	32×32 colour
Classes	10	100
Training images	50,000 (5,000 per class)	50,000 (500 per class)
Test images	10,000	10,000
Typical ResNet-18 accuracy	about 94–95%	about 75–78%

Both datasets are *balanced*: every class has the same number of training images. CIFAR-100 has ten times fewer images per class, so each individual image matters more. We should expect pruning to hurt CIFAR-100 sooner, which is one reason it is a good second test.

5.2 Four kinds of training example

Look through the training images of any real dataset and you will find roughly four kinds.

Kind	What it looks like	What happens in training
Typical (easy)	A clear, centred photo of a red car on a road. Thousands of similar images exist.	Learned in the first epoch or two and never forgotten. Loss quickly near 0.
Informative	A car photographed from an unusual angle, or a truck that looks a bit like a car.	Learned later; may flip between right and wrong a few times.
Rare but correct	One of only a handful of images of a particular breed, pose or background.	Learned late, often only by memorising it. Removing it can hurt test images of the same rare kind.
Ambiguous or mislabelled	A blurry blob; a cat labelled “dog”.	Learned last, if at all, only by memorisation. Keeping it can <i>hurt</i> test accuracy.

The trouble is that the last three kinds *all look “hard”* to a scoring method. A good pruning method must throw away the easy redundant ones and the broken ones, but keep the informative and rare ones. No score separates these perfectly.

5.3 Networks learn easy patterns first

A striking and repeatedly observed fact about neural network training is that networks fit simple, common patterns before they fit rare or noisy ones [7]. If you train on CIFAR-10 where some labels have been deliberately randomised, the network first learns the correctly labelled images; the randomised ones are fitted only later, by memorisation. This is why a model trained for only a few epochs already has an opinion about which examples are easy (it has learned them) and which are hard (it has not yet). **This is what makes early scoring possible.**

5.4 Redundancy

Many training images are near-copies of each other in the sense that matters to the network: once it has learned a hundred clear photos of red cars, the hundred-and-first adds almost nothing. These redundant images still cost as much compute as any other. Removing them is the “free lunch” part of pruning.

5.5 Memorisation and the long tail

It is tempting to conclude: “hard examples are noise, so delete them”. That is wrong in general. Real data has a *long tail*: many sub-populations each appear only a few times in the training set, and the test set contains more examples from those same rare sub-populations. Memorising the rare training examples is then the only way to get those test examples right [9]. Deleting all hard examples deletes the tail.

A “hard” example is either *valuable* (rare but correct, or close to a class boundary) or *harmful* (mislabelled or meaningless). Pruning methods that keep hard examples are betting that most hard examples are valuable. Methods that drop the very hardest are betting that the extreme end is mostly noise. Chapter 8 shows when each bet pays off.

5.6 Class balance after pruning

Different classes are not equally hard: on CIFAR-10, cats and dogs are commonly confused with each other, while cars and ships are easier. If we keep “the hardest 30% overall”, we may keep far more cat images than car images, and in extreme cases almost remove an easy class. Two ways to handle this:

- **Global pruning:** rank all images together. Simple, and the default in several papers.
- **Class-balanced pruning:** rank within each class and keep the same fraction f of each class.

It is good practice to at least *report* how many images per class survive, and class-balanced pruning is an easy variant to try.

Check yourself

1. Why might pruning hurt CIFAR-100 sooner than CIFAR-10? (*Answer: only 500 images per class, so each image carries more unique information.*)
2. Give one reason not to delete all hard examples. (*Answer: rare but correct examples are needed for the long tail of the test set.*)

Chapter 6

Data Pruning: Definitions and Strategies

This chapter states precisely what a pruning method is, defines the three strategies we will compare, and proves why random pruning is a principled baseline.

6.1 The formal setup

Definition: score, ranking and pruned set

A *scoring function* assigns a number s_i to every training example $i = 1, \dots, N$. By convention, **larger score = harder**. Given a keep fraction $f \in (0, 1]$, let $n = \lfloor fN \rfloor$ be the number of examples to keep. A *pruning strategy* picks a set $S \subseteq \{1, \dots, N\}$ with $|S| = n$. We then train a fresh model on $\{(x_i, y_i) : i \in S\}$ only.

6.2 The three strategies

Sort the examples by score from hardest to easiest.

- **Keep hard:** S = the n examples with the *largest* scores.
- **Keep easy:** S = the n examples with the *smallest* scores.
- **Random:** S = a uniformly random subset of size n (the scores are ignored).

A common refinement of keep-hard is to first discard a small slice of the *very* hardest examples (for example the top 1–5%), which are the most likely to be mislabelled, and then keep the **next-hardest**. Paul et al. found that this “offset” helps on noisier datasets [1]. You can treat it as an optional extra.

Worked example: applying the strategies

Ten images with scores (higher = harder):

image	A	B	C	D	E	F	G	H	I	J
score	0.02	0.91	0.10	0.45	0.05	1.30	0.33	0.08	0.60	0.15

Sorted hardest to easiest: F (1.30), B (0.91), I (0.60), D (0.45), G (0.33), J (0.15), C (0.10), H (0.08), E (0.05), A (0.02).

With $f = 0.4$ we keep $n = 4$ images:

- keep hard: $\{F, B, I, D\}$;
- keep easy: $\{A, E, H, C\}$;
- keep hard with the single hardest dropped as possible noise: $\{B, I, D, G\}$;
- random: for example $\{C, D, H, J\}$, a different set for each random seed.

6.3 Why random pruning is a serious baseline

It is tempting to think random pruning is a strawman. It is not; it is often surprisingly hard to beat, especially when the keep fraction is small. There is a clean mathematical reason.

Theorem 6.1 (A random subset is unbiased). *Let S be a uniformly random subset of size n from $\{1, \dots, N\}$, and let $\ell_1, \dots, \ell_N$ be the per-example losses of any fixed model. Then*

$$\mathbb{E} \left[\frac{1}{n} \sum_{i \in S} \ell_i \right] = \frac{1}{N} \sum_{i=1}^N \ell_i.$$

That is, the average loss on a random subset is, on average, exactly the average loss on the full training set. The same holds for gradients.

Proof. Write the subset sum with indicator variables: let $I_i = 1$ if $i \in S$ and 0 otherwise. Then $\sum_{i \in S} \ell_i = \sum_{i=1}^N I_i \ell_i$. By symmetry every example has the same chance of being chosen, and exactly n are chosen, so $\Pr(I_i = 1) = n/N$ and $\mathbb{E}[I_i] = n/N$. By linearity of expectation,

$$\mathbb{E} \left[\frac{1}{n} \sum_{i=1}^N I_i \ell_i \right] = \frac{1}{n} \sum_{i=1}^N \frac{n}{N} \ell_i = \frac{1}{N} \sum_{i=1}^N \ell_i. \quad \square$$

In plain words

Training on a random subset optimises (on average) the *same objective* as training on everything: it is just noisier, because there is less data. Keep-hard and keep-easy subsets are *biased*: they change which objective you are optimising. That bias is intended (we hope to spend effort where it matters), but it can go wrong, and random pruning has no bias at all to go wrong with.

Random pruning is mandatory

A result like “keeping the hardest 50% loses only 0.5% accuracy” means nothing until you know what random 50% gives. If random 50% also loses only 0.6%, the scores added almost nothing. Always run random pruning at every keep fraction, with the same number of seeds.

6.4 Static versus dynamic pruning

- **Static pruning** (our project): compute scores once, prune once, then train the final model on the fixed subset.
- **Dynamic pruning**: re-evaluate during training which examples to use in each epoch. Can be more effective but is more complicated and its cost is harder to account for.

We study static pruning because it cleanly separates “scoring cost” from “training cost”.

6.5 Two ways to train on less data: equal epochs or equal steps

When the pruned set has $n = fN$ examples, how long should we train?

- **Equal epochs**: the same number of epochs E as the full run. Each epoch is f times shorter, so the run costs about f times as much. This is what saves compute.
- **Equal steps**: the same number of gradient steps as the full run, i.e. E/f epochs. The run costs about the same as full training, so it saves nothing. It is useful for asking “are these examples as *informative* as the full set?”

Some papers use one convention and some the other. For our cost question, equal epochs is the natural choice, and it is essential to **state which one you use**. Chapter 10 returns to this.

Check yourself

1. In the worked example, what is the keep-easy set for $f = 0.2$? (Answer: $\{A, E\}$.)
2. Does Theorem 6.1 say that a random subset gives the *same* model as the full data? (Answer: no, only that the objective is the same on average; with less data the model is usually a little worse.)

Chapter 7

Scoring Examples: EL2N, GraNd and Forgetting

This chapter defines the scores we use to rank training examples, works through them numerically, and proves how they are related. Throughout, $p(x)$ is the model's softmax output on image x and y is the one-hot label.

7.1 The GraNd score: how hard does this example push?

Chapter 3 showed that the decrease in loss from one gradient step is proportional to the squared length of the gradient. So an example whose loss gradient is tiny contributes almost nothing to learning. That suggests scoring each example by the length of its gradient.

Definition: GraNd (Gradient Normed)

Train a model for t epochs; call its weights w_t . The GraNd score of example (x, y) is

$$\text{GraNd}_t(x, y) = \mathbb{E} \|\nabla_w \ell(x, y; w_t)\|_2,$$

where the expectation is over random initialisations; in practice, the average over a few independently trained models [1].

GraNd is expensive: it needs a separate gradient with respect to all 11 million weights *for every single example*. Fortunately there is a much cheaper score that ranks examples in nearly the same way.

7.2 The EL2N score: the length of the error vector

Definition: EL2N (Error L2-Norm)

$$\text{EL2N}_t(x, y) = \mathbb{E} \|p(x; w_t) - y\|_2 = \mathbb{E} \sqrt{\sum_{k=1}^K (p_k - y_k)^2}.$$

Again the expectation is estimated by averaging over a few models trained from different

random seeds.

EL2N needs only one ordinary forward pass per example, which costs about the same as one third of a training epoch (forward only, no backward). It is the main score in our project.

Worked example: three EL2N scores

Classes (cat, dog, car); all three images are labelled cat, $\mathbf{y} = (1, 0, 0)$, except the last.

Image	Prediction $\mathbf{p}$	$\mathbf{p} - \mathbf{y}$	EL2N
clear cat	(0.989, 0.007, 0.004)	(−0.011, 0.007, 0.004)	0.013
ordinary cat	(0.659, 0.242, 0.099)	(−0.341, 0.242, 0.099)	0.430
same image, labelled dog	(0.659, 0.242, 0.099)	(0.659, −0.758, 0.099)	1.009

For the middle row:

$$\sqrt{0.341^2 + 0.242^2 + 0.099^2} = \sqrt{0.1163 + 0.0586 + 0.0097} = \sqrt{0.1846} = 0.430.$$

The mislabelled image gets by far the highest score. This is why the very top of an EL2N ranking tends to contain label errors, and why looking at the top-scored images is a good sanity check.

7.3 Why EL2N approximates GraNd: a derivation

We now show that the gradient with respect to the weights is “the error vector $\mathbf{p} - \mathbf{y}$, passed backwards through the network”.

Step 1: the chain rule through the logits. The loss depends on the weights only through the logits $z_1, \dots, z_K$. So

$$\nabla_{\mathbf{w}} \ell = \sum_{k=1}^K \frac{\partial \ell}{\partial z_k} \nabla_{\mathbf{w}} z_k = \sum_{k=1}^K (p_k - y_k) \nabla_{\mathbf{w}} z_k,$$

using Theorem 4.3. In matrix form, $\nabla_{\mathbf{w}} \ell = J^\top (\mathbf{p} - \mathbf{y})$, where J is the $K \times d$ matrix whose k -th row is $\nabla_{\mathbf{w}} z_k$ (the *Jacobian* of the logits).

Step 2: an upper bound. Let $\|J\|_{\text{op}}$ be the largest factor by which $J^\top$ can stretch a vector (its *operator norm*, Appendix A). Then

$$\|\nabla_{\mathbf{w}} \ell\|_2 = \|J^\top (\mathbf{p} - \mathbf{y})\|_2 \leq \|J\|_{\text{op}} \|\mathbf{p} - \mathbf{y}\|_2. \quad (7.1)$$

So an example with a small EL2N score always has a small gradient. If $\|J\|_{\text{op}}$ is similar across examples, GraNd and EL2N rank examples similarly.

Step 3: an exact case, the last layer. For the final linear layer, $\mathbf{z} = W\mathbf{a} + \mathbf{b}$, where $\mathbf{a} \in \mathbb{R}^m$ is the feature vector from the penultimate layer and W is a $K \times m$ weight matrix. Then $\partial z_k / \partial W_{kj} = a_j$, so

$$\frac{\partial \ell}{\partial W_{kj}} = (p_k - y_k) a_j, \quad \text{i.e.} \quad \nabla_W \ell = (\mathbf{p} - \mathbf{y}) \mathbf{a}^\top.$$

Lemma 7.1. For vectors $\mathbf{u} \in \mathbb{R}^K$ and $\mathbf{v} \in \mathbb{R}^m$, the matrix $\mathbf{u}\mathbf{v}^\top$ has Frobenius norm $\|\mathbf{u}\mathbf{v}^\top\|_F = \|\mathbf{u}\|_2 \|\mathbf{v}\|_2$.

Proof. $\|\mathbf{u}\mathbf{v}^\top\|_F^2 = \sum_{k,j} (u_k v_j)^2 = (\sum_k u_k^2) (\sum_j v_j^2) = \|\mathbf{u}\|_2^2 \|\mathbf{v}\|_2^2.$ $\square$

Therefore the last-layer gradient norm is *exactly*

$$\|\nabla_w \ell\|_F = \underbrace{\|\mathbf{p} - \mathbf{y}\|_2}_{\text{EL2N}} \times \underbrace{\|\mathbf{a}\|_2}_{\text{feature size}}.$$

If feature vectors have roughly similar lengths across images (batch normalisation encourages this), the last-layer GraNd is essentially EL2N times a constant.

7.4 How EL2N relates to confidence: a two-sided bound

EL2N depends on the whole probability vector, but it is pinned down closely by one number: the probability given to the correct class, p_y .

Theorem 7.2 (EL2N is sandwiched by $1 - p_y$). *For any probability vector $\mathbf{p}$ and one-hot label $\mathbf{y}$ with true class y ,*

$$1 - p_y \leq \|\mathbf{p} - \mathbf{y}\|_2 \leq \sqrt{2}(1 - p_y).$$

In particular $0 \leq \text{EL2N} \leq \sqrt{2}$.

Proof. Split the squared norm into the true-class entry and the rest:

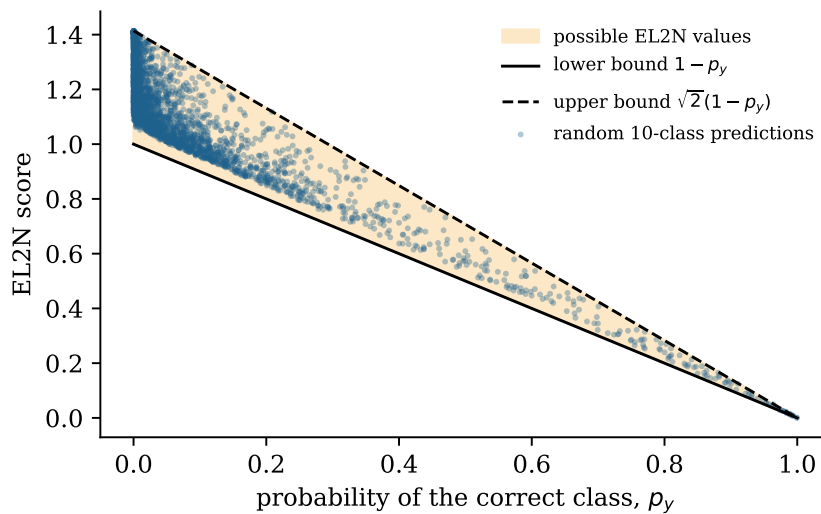
$$\|\mathbf{p} - \mathbf{y}\|_2^2 = (p_y - 1)^2 + \sum_{j \neq y} p_j^2.$$

Lower bound. The sum is non-negative, so $\|\mathbf{p} - \mathbf{y}\|_2^2 \geq (1 - p_y)^2$.

Upper bound. All $p_j \geq 0$, so the sum of squares is at most the square of the sum (expanding $(\sum_j p_j)^2$ gives the squares plus non-negative cross terms):

$$\sum_{j \neq y} p_j^2 \leq \left(\sum_{j \neq y} p_j \right)^2 = (1 - p_y)^2,$$

because the probabilities add to 1. Hence $\|\mathbf{p} - \mathbf{y}\|_2^2 \leq 2(1 - p_y)^2$. Taking square roots gives both bounds. Since $0 \leq p_y \leq 1$, the maximum is $\sqrt{2}$. $\square$



Every one of 3,000 randomly generated 10-class predictions lies between the two bounds of Theorem 7.2.

Consequence. Ranking by EL2N is almost the same as ranking by $1 - p_y$, and therefore almost the same as ranking by the cross-entropy loss $-\log p_y$ (which also decreases as p_y increases). EL2N, loss and confidence are three views of one idea: *how far is the model from being confidently right on this example?*

7.5 Averaging over several scorer runs

A single trained model gives noisy scores: an example that happens to be learned early in one run may be learned late in another. Paul et al. therefore average EL2N over several independently trained models. How much does that help?

Proposition 7.3. Let $s^{(1)}, \dots, s^{(k)}$ be the scores of one example from k independently trained models, each with mean μ and variance σ^2 . Their average $\bar{s} = \frac{1}{k} \sum_r s^{(r)}$ has mean μ and variance σ^2/k .

Proof. The mean follows from linearity of expectation. For the variance, independence means variances add, and scaling by $1/k$ scales the variance by $1/k^2$:

$$\text{Var}(\bar{s}) = \frac{1}{k^2} \sum_{r=1}^k \text{Var}(s^{(r)}) = \frac{1}{k^2} k\sigma^2 = \frac{\sigma^2}{k}. \quad \square$$

So averaging 4 runs halves the noise (standard deviation $\sigma/\sqrt{4}$), but **costs 4 times as much**. In our project this trade-off is central: more scorer runs give better scores but eat into the savings. A cheap scorer lets us afford more runs.

7.6 The forgetting score

Toneva et al. [4] score examples by watching them during training.

Definition: forgetting event

Let $a_i^{(t)} = 1$ if example i is classified correctly at the t -th time it is seen during training, and 0 otherwise. A *forgetting event* happens when $a_i^{(t)} = 1$ and $a_i^{(t+1)} = 0$: the example was right and then became wrong. The *forgetting score* of example i is the number of forgetting events it has during training. Examples that are never learned are given the maximum possible score, since they are the hardest of all.

Worked example: counting forgetting events

An image's correctness over 10 epochs:

0, 1, 1, 0, 1, 1, 1, 0, 0, 1.

Look for the pattern “1 then 0”: it occurs at epochs 3→4 and 7→8. **So the forgetting score is 2.** An image with the sequence 0, 1, 1, 1, 1, 1, 1, 1, 1, 1 has **score 0: it is unforgettable.**

Toneva et al. found that a large fraction of CIFAR-10 images are unforgettable and that many of them can be removed with little loss in accuracy. The forgetting score is cheap to record (you

get it from predictions you already make during training), but it needs a *long* training run to be meaningful, so as a pruning score it is expensive.

7.7 Comparing the scores

	EL2N	GraNd	Forgetting
Measures	length of the error vector $p - y$	length of the weight gradient	number of right-to-wrong flips
Needs	one forward pass after t epochs	a backward pass per example	correctness at every epoch
Extra cost	small	large	small to record, but needs a long training run
Works early?	yes, after a few epochs	yes	no, needs many epochs
Our use	main score	reference only	second score, for comparison

How to compute EL2N correctly

- Put the model in evaluation mode (`model.eval()`) so batch normalisation uses its stored statistics.
- Turn *off* data augmentation: score the plain image.
- Do not shuffle, or keep the index of every image, so that scores line up with image IDs.
- Save the scores to disk immediately (e.g. a `.npy` file). Recomputing them is exactly the cost we are trying to avoid.

Check yourself

1. A model gives the true class probability 0.9. Between which two values must EL2N lie? (*Answer: 0.1 and 0.141.*)
2. Count the forgetting events in 1,0,1,0,1,0,1,1. (*Answer: 3.*)
3. How many scorer runs are needed to reduce the score noise to one third? (*Answer: $k = 9$.*)

Chapter 8

Why the Best Strategy Depends on How Much You Keep

This is the most “theoretical” chapter, but everything in it can be followed with school algebra and one integral. We study two small problems in which we can see exactly why keeping hard examples can be very good, why it can fail, and why the quality of the scorer matters most when we prune aggressively. The second of these failure modes is the core risk of a cheap scorer.

8.1 A toy problem you can solve by hand

Setup. Inputs are single numbers x drawn uniformly from $[0, 1]$. There is an unknown threshold θ (in the figures, $\theta = 0.3$): the label is “positive” if $x > \theta$ and “negative” otherwise. We have a pool of $N = 1000$ labelled points.

Learner. Given some training points, the learner finds the largest negative point ℓ and the smallest positive point h , and guesses the threshold halfway between them:

$$\hat{\theta} = \frac{\ell + h}{2}.$$

(If there are no negative points it uses $\ell = 0$; if there are no positive points it uses $h = 1$.) This is the “maximum margin” rule: it puts the boundary as far as possible from the training points on both sides.

Proposition 8.1. *The test error of the learner (the chance of misclassifying a new uniform point) is exactly $|\hat{\theta} - \theta|$.*

Proof. A new point x is misclassified exactly when it lies between θ and $\hat{\theta}$ (the true and guessed boundaries disagree there, and nowhere else). For a uniform point on $[0, 1]$, the probability of landing in an interval is its length, $|\hat{\theta} - \theta|$. $\square$

Scores. A point close to θ is “hard” (it is near the boundary) and a point far away is “easy”. A perfect scorer knows θ and gives each point the score $-|x - \theta|$ (closer = harder).

8.2 Random data: error shrinks like $1/n$

The learner's mistake comes from the empty gaps on each side of θ . Let G be the distance from θ to the nearest positive training point.

Theorem 8.2. *If the training set has n points drawn independently and uniformly from $[0, 1]$,*

$$\mathbb{E}[G] = \frac{1 - \theta^{n+1}}{n+1} \approx \frac{1}{n+1}.$$

Proof. For $0 \leq t \leq 1 - \theta$, the event $G > t$ means that no training point falls in the interval $(\theta, \theta + t]$. Each point avoids it with probability $1 - t$, independently, so $\Pr(G > t) = (1 - t)^n$. For a non-negative random variable, $\mathbb{E}[G] = \int_0^\infty \Pr(G > t) dt$ (Appendix A). G never exceeds $1 - \theta$ (that is the value used when there are no positive points), so

$$\mathbb{E}[G] = \int_0^{1-\theta} (1 - t)^n dt = \left[-\frac{(1 - t)^{n+1}}{n+1} \right]_0^{1-\theta} = \frac{1 - \theta^{n+1}}{n+1}. \quad \square$$

Worked example: checking Theorem 8.2 by simulation

With $\theta = 0.3$, averaging G over 20,000 random training sets:

n	5	10	20	50	100	200
simulated $\mathbb{E}[G]$	0.1666	0.0908	0.0478	0.0196	0.0099	0.0050
formula	0.1667	0.0909	0.0476	0.0196	0.0099	0.0050

The same argument applies to the gap G' on the negative side. The error is $|\hat{\theta} - \theta| = |G - G'|/2 \leq (G + G')/2$, so the expected error is at most about $1/(n+1)$: **it shrinks like $1/n$** . Doubling the data halves the error. This kind of steady “power-law” improvement is what we usually see in real deep learning too: to halve the error you need several times more data.

Random pruning to a fraction f gives a random training set of $n = fN$ points, so its error is about $1/(fN)$. Keeping 10% of the data makes the error about 10 times larger.

8.3 Keeping hard examples with a perfect scorer

Now keep the n points *closest* to θ .

Proposition 8.3. *If the n kept points include at least one point on each side of θ , the learner's output is identical to what it would output using all N points.*

Proof. The learner only uses ℓ (the negative point closest to θ) and h (the positive point closest to θ). In the full pool, these are the closest points to θ on each side. The kept set is “all points within some distance r of θ ”, so if it contains any point on a given side, it contains the closest one on that side. So ℓ and h are the same as for the full pool. $\square$

When could the kept set miss a side entirely? Only if all n closest points lie on the same side. Near θ , each point is equally likely to be on either side, so this has probability about $2 \cdot 2^{-n}$, which is less than one in a thousand once $n \geq 11$.

With a perfect scorer, keeping only the hard examples gives the *full-data* error, about $1/N$, no matter how small n is. Random pruning gives about $1/n$. In this toy, hard-example pruning turns a small dataset into one that behaves like a large one. This is the idea behind the title *Beyond Neural Scaling Laws* [2]: with a very good score, error can fall much faster with data than the usual power law.

8.4 Keeping hard examples with an *imperfect* scorer

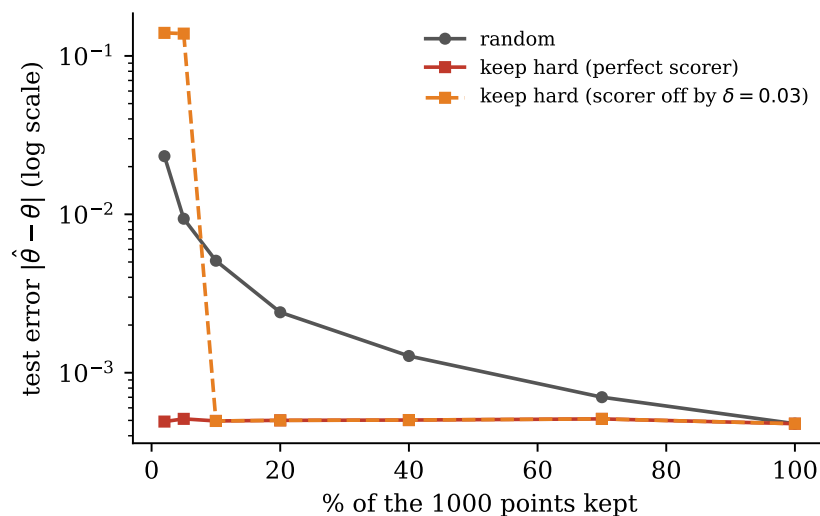
Now suppose the scorer is slightly wrong: it believes the boundary is at $\theta + \delta$ instead of θ (for example, a cheap model that has learned a slightly wrong boundary). It keeps the n points closest to $\theta + \delta$.

The pool has N points spread evenly, so $n = fN$ points occupy a window of width about f . The kept window is therefore about

$$\left[\theta + \delta - \frac{f}{2}, \theta + \delta + \frac{f}{2} \right].$$

Theorem 8.4 (How good must the scorer be?). *In this toy, hard pruning with a scorer that is off by $\delta > 0$ recovers the full-data error when $f > 2\delta$ (the window reaches past the true boundary). When $f < 2\delta$ the kept window contains no negative points at all, the learner cannot see the boundary, and the error jumps to roughly $(\theta - \delta + f/2)/2$, a large error that does not shrink with more data (about 0.14 in our example).*

Explanation. If $\delta < f/2$, the lower edge of the window, $\theta + \delta - f/2$, is below θ , so the window contains points on both sides of the true boundary, and the previous proposition applies. If $\delta > f/2$, the lower edge is above θ , so every kept point is positive. The learner then uses $\ell = 0$ and $h \approx \theta + \delta - f/2$, so $\hat{\theta} \approx (\theta + \delta - f/2)/2$, and the error is $\theta - \hat{\theta} \approx (\theta - \delta + f/2)/2$. With $\theta = 0.3$, $\delta = 0.03$ and $f = 0.02$: $h \approx 0.32$, $\hat{\theta} \approx 0.16$, and the error is about 0.14. (At $f = 0.05$ the formula gives 0.148; the simulation gives 0.138 because the window occasionally reaches just below θ by chance.) $\square$



The toy problem with $N = 1000$, $\theta = 0.3$, averaged over 2,000 trials. A perfect hard-example scorer (red) matches full-data error at every keep fraction. A scorer that is off by only $\delta = 0.03$ (orange, dashed) is

equally good when keeping 10% or more, but fails badly at 2% and 5%, exactly as Theorem 8.4 predicts ($2\delta = 6\%$). Random pruning (grey) degrades smoothly like $1/n$.

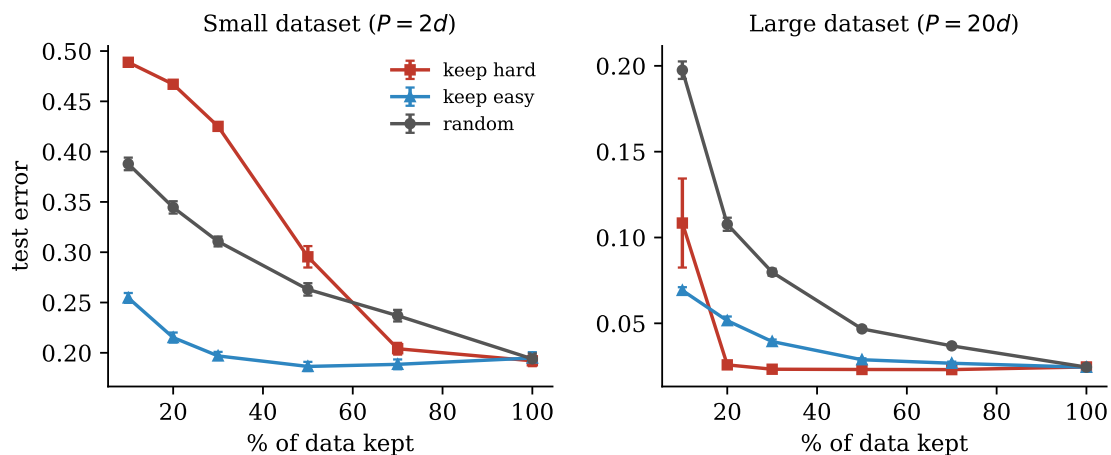
The lesson for our project. The allowed error in the scorer is proportional to how much data we keep. Mild pruning (keeping 50–90%) tolerates a rough, cheap scorer. Aggressive pruning (keeping 10–30%) needs an accurate scorer, and a cheap scorer is most likely to break exactly where the potential savings are largest. We should therefore expect a cheap scorer to be “good enough” at high keep fractions and to fall behind at low ones. Measuring where that happens is one of our main results.

8.5 Beyond one dimension: when keeping easy examples wins

The 1D toy cannot show one important effect: *with very little data, keeping the easy examples can be better than keeping the hard ones*. To see it we need more dimensions. Sorscher et al. [2] analysed a “teacher–student” model; we reproduce its spirit in a small simulation.

Setup.

- Inputs x are vectors of $d = 50$ random numbers (standard normal).
- A hidden “teacher” direction t defines the labels: $y = \text{sign}(t \cdot x)$.
- The score of an example is its *margin* $|t \cdot x|$: small margin = close to the boundary = hard.
- A “student” (a maximum-margin linear classifier, i.e. a linear SVM) is trained on the kept examples. Its test error is the angle between student and teacher directions divided by π .
- We use a small pool ($P = 2d = 100$ examples) and a large pool ($P = 20d = 1000$), keep fractions from 10% to 100%, and average 20 trials.



Test error (mean $\pm$ standard error over 20 trials) in the teacher–student simulation with a perfect scorer.

What we see.

- **Small pool (left):** keeping the *easy* examples is best at every keep fraction (for example, at 10% kept: easy 0.26, random 0.39, hard 0.49). Keeping only hard examples is *worse than random*.

- **Large pool** (right): keeping the *hard* examples is best from 20% upwards (at 20%: hard 0.026, easy 0.052, random 0.108). Hard pruning at 20% already matches full-data error.
- At 10% of the large pool (just 100 examples = $2d$), easy wins again (0.069 vs. 0.108 for hard): with 100 examples we are back in the “small data” situation.

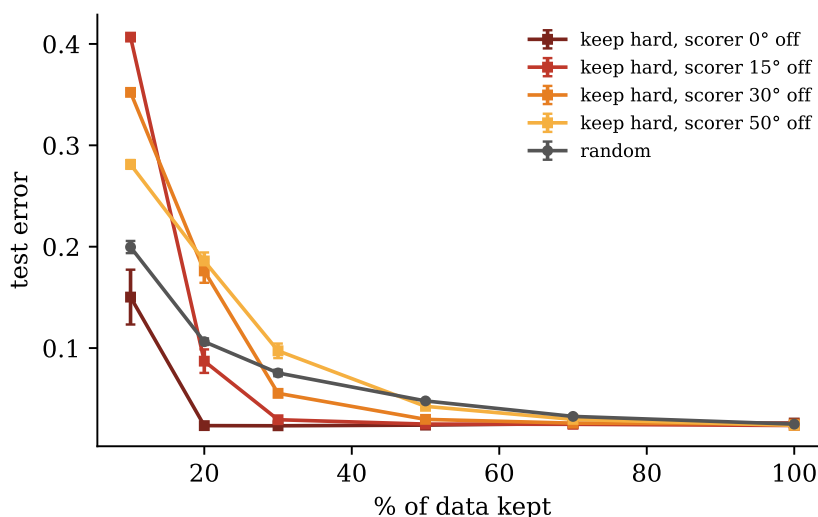
Why the crossover happens

With very few examples, the student does not yet know even the rough direction of the boundary. Easy examples, far from the boundary, point clearly to which side is which, and they are the quickest way to learn the rough direction. Hard examples, right at the boundary, are only useful for *fine-tuning* a direction the student already roughly knows; on their own they are confusing. With plenty of examples the rough direction is easy to get, and the hard ones supply the fine detail. So **keep easy when you keep little; keep hard when you keep a lot**.

Sorscher et al. derive this crossover mathematically for the teacher–student model and confirm it on real image datasets [2]. Reproducing a version of it on CIFAR-10 (hard vs. easy vs. random at several keep fractions) is a natural mid-project result.

8.6 The imperfect scorer in the teacher–student model

Finally, we repeat the large-pool experiment with scorers that use a slightly wrong direction: rotated 15° , 30° or 50° away from the teacher. These play the role of cheap scorers.



Keep-hard pruning with increasingly inaccurate scorers (large pool, 20 trials).

At 50% kept, all scorers are close to full-data error (between 0.024 and 0.043). At 20% kept, the perfect scorer gives 0.023 but the 30° scorer gives 0.176, *worse than random* (0.106). This is the same message as Theorem 8.4: **an imperfect scorer is fine for mild pruning and dangerous for aggressive pruning**.

Check yourself

1. In the 1D toy, a scorer is off by $\delta = 0.1$. What is the smallest keep fraction at which hard pruning still works? (*Answer: about $f = 2\delta = 20\%$.)*)
2. With random pruning, going from $n = 100$ to $n = 400$ points changes the 1D error by what factor? (*Answer: about $1/4$.)*)
3. You have a small dataset and must keep only 10%. Which strategy does the teacher–student model suggest? (*Answer: keep easy.*)

Chapter 9

Cheap Scorers: Can a Small Model Choose for a Big One?

This chapter is about the first question of our project. We explain why a small model might (or might not) be a good judge of which examples a big model needs, and how to measure how well two scorers agree.

9.1 The idea of a proxy

A *proxy* is a stand-in: something cheap that we use in place of something expensive. Coleman et al. [3] proposed *Selection via Proxy*: to select data for a large model, use a smaller model, or a model trained for fewer epochs, to do the selection. They reported that proxies often chose data almost as well as the full model, at a fraction of the cost. Our project asks the same question specifically for EL2N-based pruning on CIFAR, and adds a careful count of the total cost.

In our setting a scorer has three “cheapness knobs”:

Knob	Cheap setting	Expensive setting
Scorer size	tiny CNN	ResNet-18 (same as the target)
Scorer training length	1 epoch	20 epochs
Number of scorer runs averaged	1	3–5

9.2 Reasons to expect a cheap scorer to work

- **Difficulty is partly a property of the data.** A blurry image, a mislabelled image or an unusual pose is hard for any model. Many easy images are easy for every model.
- **All networks learn easy patterns first** (Chapter 5). After a few epochs, even a small network has sorted the obvious cases from the rest.
- **Mild pruning is forgiving** (Chapter 8). When we keep 70–90% of the data, even a rough score mostly removes redundant easy images.

9.3 Reasons to expect it to fail

- **Capacity mismatch.** A small network finds many examples hard that ResNet-18 finds easy (it simply cannot represent the relevant features). Its “hardest” list is then full of examples that ResNet-18 would learn anyway, while the examples that are genuinely hard for ResNet-18 may be ranked in the middle.
- **Boundary mismatch.** In the language of Chapter 8, the small model’s decision boundary is in a slightly different place. Keeping “the examples near *its* boundary” is then like the scorer that is off by δ : fine at high keep fractions, damaging at low ones.
- **Noise.** A briefly trained, small model gives noisier scores, and noise in the ranking behaves like a less accurate scorer.

Our hypothesis. A cheap scorer will be about as good as the expensive one when we keep a large fraction of the data, and will fall behind (possibly below random) when we keep a small fraction. The interesting number is the keep fraction at which it starts to fail, and whether the compute saved above that point is worth it.

9.4 Measuring agreement between two scorers

Before training anything on the pruned data, we can directly compare the rankings produced by two scorers (for example, tiny CNN vs. ResNet-18). Three measures are useful.

9.4.1 Spearman rank correlation

Replace each score by its *rank* (1 = hardest, N = easiest). Let r_i and s_i be the ranks of example i under scorers A and B, and $d_i = r_i - s_i$.

Definition: Spearman’s ρ

Spearman’s ρ is the ordinary (Pearson) correlation between the two lists of ranks. When there are no ties it equals

$$\rho = 1 - \frac{6 \sum_{i=1}^N d_i^2}{N(N^2 - 1)}.$$

$\rho = 1$ means identical rankings, $\rho = -1$ means exactly reversed, and $\rho \approx 0$ means unrelated.

Theorem 9.1. *Without ties, the Pearson correlation of the ranks equals $1 - \frac{6 \sum_i d_i^2}{N(N^2 - 1)}$.*

Proof. Both rank lists are rearrangements of $1, 2, \dots, N$, so they have the same mean and variance. Using the sums $\sum_{k=1}^N k = \frac{N(N+1)}{2}$ and $\sum_{k=1}^N k^2 = \frac{N(N+1)(2N+1)}{6}$ (Appendix A):

$$\begin{aligned} \text{mean} &= \frac{N+1}{2}, \\ \text{variance} &= \frac{1}{N} \sum_k k^2 - \left(\frac{N+1}{2} \right)^2 = \frac{(N+1)(2N+1)}{6} - \frac{(N+1)^2}{4} = \frac{N^2 - 1}{12}. \end{aligned}$$

Next, expanding $d_i^2 = r_i^2 + s_i^2 - 2r_i s_i$ and summing,

$$\sum_i r_i s_i = \frac{1}{2} \left(\sum_i r_i^2 + \sum_i s_i^2 - \sum_i d_i^2 \right) = \frac{N(N+1)(2N+1)}{6} - \frac{1}{2} \sum_i d_i^2.$$

The covariance of the ranks is therefore

$$\frac{1}{N} \sum_i r_i s_i - \left(\frac{N+1}{2} \right)^2 = \frac{N^2 - 1}{12} - \frac{1}{2N} \sum_i d_i^2,$$

using the same simplification as for the variance. Dividing the covariance by the variance (both lists have variance $\frac{N^2-1}{12}$):

$$\rho = 1 - \frac{\sum_i d_i^2 / (2N)}{(N^2 - 1) / 12} = 1 - \frac{6 \sum_i d_i^2}{N(N^2 - 1)}.$$

□

9.4.2 Kendall's tau

Definition: Kendall's τ

Look at every pair of examples (i, j) . The pair is *concordant* if both scorers put them in the same order and *discordant* otherwise. With C concordant and D discordant pairs out of $N(N-1)/2$,

$$\tau = \frac{C - D}{N(N-1)/2}.$$

Kendall's τ has a direct interpretation: pick two examples at random; $\frac{1+\tau}{2}$ is the probability that the two scorers agree on which of them is harder.

Worked example: Spearman and Kendall on five images

image	P	Q	R	S	T
rank by ResNet-18 (r)	1	2	3	4	5
rank by tiny CNN (s)	2	1	3	5	4
$d = r - s$	-1	1	0	-1	1

Spearman: $\sum d^2 = 4$, so $\rho = 1 - \frac{6 \cdot 4}{5 \cdot 24} = 1 - 0.2 = 0.8$.

Kendall: of the 10 pairs, only (P,Q) and (S,T) are in the opposite order, so $C = 8$, $D = 2$ and $\tau = (8 - 2)/10 = 0.6$. Picking two images at random, the scorers agree on which is harder with probability 0.8.

9.4.3 Overlap of the kept sets

Correlations look at the whole ranking, but pruning only cares about *which examples end up kept*. The most directly relevant measure is:

Definition: kept-set overlap

If scorers A and B each keep n examples, forming sets S_A and S_B , their overlap is $|S_A \cap S_B|/n$, the fraction of A's kept examples that B also keeps.

Proposition 9.2. If S_A is any fixed set of size $n = fN$ and S_B is a uniformly random set of size n , the expected overlap is f .

Proof. Each example in S_A belongs to S_B with probability $n/N = f$ (the same indicator argument as Theorem 6.1). By linearity of expectation, $\mathbb{E}|S_A \cap S_B| = nf$, so the expected overlap is f . $\square$

Compare overlap with the random baseline

When keeping 90%, two *random* sets already overlap by about 90%. An overlap of 92% is therefore almost no agreement beyond chance. At 30% kept, random overlap is 30%, so an overlap of 70% is strong agreement. Always report the overlap next to f .

9.5 The experiments that answer the question

For each scorer setting we record:

1. its **cost** (GPU seconds for training the scorer(s) plus computing scores);
2. its **agreement** with the reference scorer (ResNet-18, several epochs, averaged over runs): Spearman ρ and the kept-set overlap at each keep fraction;
3. most importantly, the **test accuracy of ResNet-18 trained on the data it selects**, at each keep fraction, compared with random pruning.

Scorer	Epochs	Runs	Cost (s)	ρ vs. reference	Acc. at $f = 0.5 / 0.3$
tiny CNN	1, 5, 20	3	...	...	...
ResNet-18	1, 5, 20	3	...	...	...
(random)	–	–	0	–	...

The agreement numbers are cheap to compute and help explain the accuracy results: if the tiny CNN's list overlaps strongly with ResNet-18's at $f = 0.5$ but weakly at $f = 0.3$, that explains why it works at one and not the other.

Check yourself

1. Two rankings of 10 examples have $\sum d_i^2 = 33$. What is ρ ? (Answer: $1 - 198/990 = 0.8$.)
2. At $f = 0.5$, is an overlap of 55% impressive? (Answer: no, random sets overlap by 50% on average.)

Chapter 10

Honest Compute Accounting

This chapter answers the question “does pruning actually save time?” with a simple cost model. It is the part of the project that most published work skips, and it is our main contribution.

10.1 What to measure

We measure **GPU wall-clock time**: the number of seconds a job takes on the GPU. Two practical rules:

- GPU operations run asynchronously, so call `torch.cuda.synchronize()` before reading the clock at the start and end (the code in Chapter 13 does this).
- Compare timings only on the *same GPU type*. A T4 and a P100 differ a lot. Record the GPU name with every result.

It is also useful to know the *theoretical* work (FLOPs, from equation (4.1)), but wall-clock time is what the project question is about. They can differ: a tiny network may leave the GPU idle while it waits for the data loader, so its real speed-up over ResNet-18 can be smaller than its FLOP ratio suggests.

10.2 The cost model

Let F be the cost of one ordinary full-data training run of the target model (ResNet-18, E epochs, all N images).

Training on the pruned set. With equal epochs, the pruned run processes f times as many images (Chapter 3), so it costs about fF .

Scoring. Suppose we train k scorer models, each for e epochs, and each is r times as expensive per image as the target model ($r = 1$ if the scorer is a ResNet-18, $r < 1$ for a tiny CNN). After training, each scorer does one forward pass over the N images, which costs about one third of an epoch (forward only, no backward pass). So

$$S = k \cdot r \cdot \left(e + \frac{1}{3}\right) \cdot \frac{F}{E}. \quad (10.1)$$

Total cost of pruning.

$$C_{\text{prune}} = S + fF. \quad (10.2)$$

Theorem 10.1 (Break-even condition). *Pruning (with equal epochs) is cheaper than full training if and only if*

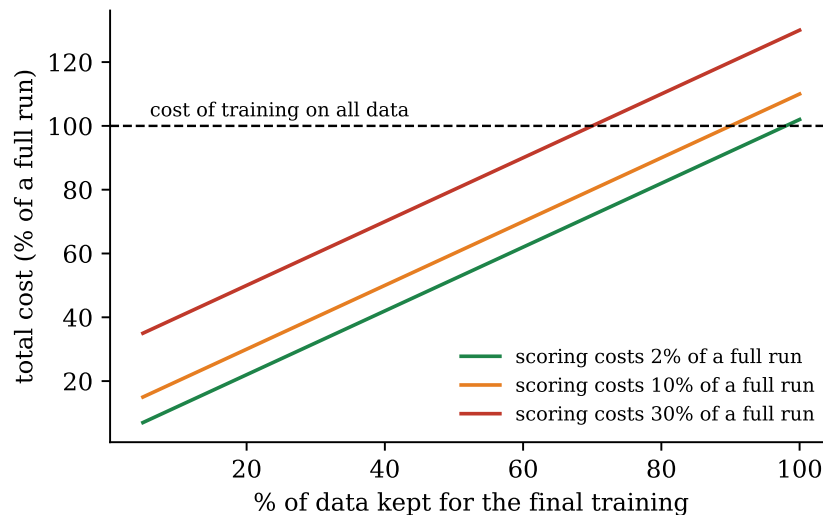
$$f < 1 - \frac{S}{F}.$$

The resulting speed-up is $\frac{F}{S + fF} = \frac{1}{S/F + f}$.

Proof. $S + fF < F \iff fF < F - S \iff f < 1 - S/F$ (dividing by $F > 0$). The speed-up is the ratio of costs, $F/(S + fF)$; divide top and bottom by F . $\square$

In plain words

The scoring cost, as a fraction of a full run, is a “tax” of S/F . You only profit if the fraction of data you throw away, $1 - f$, is larger than the tax.



Total cost $S + fF$ for three scoring costs. Pruning saves compute only where a line is below the dashed line.

10.3 Worked examples

The numbers below are *illustrative*: they show how to use the model. Your project will replace them with measured values.

Worked example: self-scoring with ResNet-18

Full training: $E = 60$ epochs. Score with ResNet-18 itself ($r = 1$), $k = 3$ runs of $e = 10$ epochs:

$$\frac{S}{F} = \frac{3 \cdot 1 \cdot (10 + 1/3)}{60} = 0.517.$$

The break-even keep fraction is $1 - 0.517 = 0.483$. Keeping 50% therefore costs $0.517 + 0.5 = 1.017$ full runs, *more* than just training on everything. Keeping 30% costs 0.817 full

runs, a speed-up of only $1/0.817 = 1.22\times$, and at 30% we may well have lost accuracy.

Worked example: a cheap scorer

Now score with a tiny CNN that is 10 times cheaper per image ($r = 0.1$), $k = 3$ runs of $e = 5$ epochs:

$$\frac{S}{F} = \frac{3 \cdot 0.1 \cdot (5 + 1/3)}{60} = 0.027.$$

Keeping 50% costs 0.527 full runs (speed-up $1.90\times$); keeping 30% costs 0.327 (speed-up $3.06\times$). The scoring tax has almost disappeared, so the question becomes purely whether the accuracy holds up.

These two examples are the project in miniature: **the expensive scorer gives good scores but eats the savings; the cheap scorer is nearly free but may give worse scores.**

10.4 The baseline that is easy to forget

There is another cheap way to spend fF of compute: **train on all the data for fE epochs** (a shorter schedule). This costs exactly the same as training on the pruned set for E epochs.

Compare against the equal-cost full-data run

If ResNet-18 trained on all 50,000 images for 30 epochs (with a correctly shortened learning-rate schedule) beats ResNet-18 trained on a pruned 25,000 images for 60 epochs, then pruning is not the best use of that compute. Include this “short schedule” baseline in the cost table; it makes the comparison honest.

10.5 Amortisation: when the pruned set is reused

Scoring is done once, but the pruned dataset can be reused, for example to try several learning rates or architectures. If the subset is used for R training runs, the total cost of pruning is $S + RfF$ against RF without pruning. The break-even condition becomes

$$f < 1 - \frac{S}{RF},$$

so the scoring tax is divided by R . This is a legitimate argument in favour of pruning, as long as it is stated clearly. It also suggests a practical rule: *score once and share the score files*, which is exactly what the team plan in Chapter 14 does.

10.6 Equal steps: why it saves nothing

If instead we train the pruned set for E/f epochs (equal number of gradient steps), the training cost is $\frac{E}{f} \cdot fN = EN$ image-views, the same as full training. The total is then $S + F > F$: **with equal steps, pruning can only cost more.** Equal steps is still a meaningful experiment (it asks whether the kept examples are as *informative* per step as the full data), but it cannot answer the compute question. Report it separately if you run it.

10.7 The cost table we will produce

Method	f	Scoring (min)	Training (min)	Total (min)	Test acc. (%)
Full data, E epochs	1.0	0	...	...	...
Full data, fE epochs	1.0	0	...	...	...
Random pruning	...	0	...	...	...
EL2N, ResNet-18 scorer	...	...	...	...	...
EL2N, tiny-CNN scorer	...	...	...	...	...

A good way to present the result is a single scatter plot with *total cost* on the horizontal axis and *test accuracy* on the vertical axis, one point per method and keep fraction. The best methods lie towards the top-left (cheap and accurate). Any method that is both slower and less accurate than another is simply dominated.

Check yourself

1. Scoring costs $S/F = 0.2$. What is the largest keep fraction that still saves compute? What is the speed-up at $f = 0.4$? (*Answer: 0.8; $1/0.6 \approx 1.67\times$.*)
2. With the ResNet-18 self-scoring numbers above, how many reuses R make $f = 0.5$ break even? (*Answer: need $0.517/R < 0.5$, so $R \geq 2$.*)

Chapter 11

How This Relates to Knowledge Distillation

You asked whether this project is related to distilled models. This chapter explains knowledge distillation properly, derives its key formulas, and then compares it with our project.

11.1 What knowledge distillation is

In *knowledge distillation* [5] a large, accurate *teacher* model is trained first. A smaller *student* model is then trained to match not only the true labels but also the teacher's *soft predictions*: its full probability vector on every training image.

Why would soft predictions help? A one-hot label says only “this is a cat”. The teacher's probabilities also say “it looks a bit like a dog and not at all like a car”. This extra information, sometimes called *dark knowledge*, tells the student how the classes relate, and the student usually generalises better than if trained on hard labels alone.

11.2 The distillation loss

The teacher's probabilities are usually very peaked, so the useful small probabilities are hard to see. Both models' logits are therefore divided by a temperature $T > 1$ (Chapter 4):

$$\mathbf{p}_t^{(T)} = \text{softmax}(\mathbf{z}_t/T), \quad \mathbf{p}_s^{(T)} = \text{softmax}(\mathbf{z}_s/T).$$

The student is trained on a mixture of two losses:

$$L_{\text{KD}} = (1 - \alpha) \underbrace{(-\log p_{s,y})}_{\text{usual cross-entropy}} + \alpha T^2 \underbrace{\text{KL}(\mathbf{p}_t^{(T)} \parallel \mathbf{p}_s^{(T)})}_{\text{match the teacher}}, \quad (11.1)$$

where $\alpha \in [0, 1]$ balances the two and the Kullback–Leibler divergence is

$$\text{KL}(\mathbf{q} \parallel \mathbf{p}) = \sum_k q_k \log \frac{q_k}{p_k} = \underbrace{\sum_k q_k \log q_k}_{\text{does not depend on the student}} - \sum_k q_k \log p_k.$$

The KL divergence is 0 when the two distributions are equal and positive otherwise.

11.3 The gradient of the distillation term

Theorem 11.1. With $q = p_t^{(T)}$ fixed,

$$\nabla_{z_s} \text{KL}(q \parallel \text{softmax}(z_s/T)) = \frac{1}{T} (p_s^{(T)} - q).$$

Proof. Only the term $-\sum_k q_k \log p_k$ depends on the student. Let $u = z_s/T$, so $p_s^{(T)} = \text{softmax}(u)$. Repeating the proof of Theorem 4.3 with a soft target q instead of a one-hot y :

$$\frac{\partial}{\partial u_j} \left(-\sum_k q_k \log p_k \right) = -\sum_k \frac{q_k}{p_k} p_k (\delta_{kj} - p_j) = -q_j + p_j \sum_k q_k = p_j - q_j,$$

using $\sum_k q_k = 1$. Finally, by the chain rule, $\partial u_j / \partial z_{s,j} = 1/T$. $\square$

So distillation, like ordinary training, pushes the student along “prediction minus target”. The only difference is that the target is the teacher’s soft distribution.

11.4 Why the loss is multiplied by T^2

For a large temperature, $e^u \approx 1 + u$ when u is small. Assume (as softmax allows, by shift invariance) that the logits of each model have mean zero, and write K for the number of classes. Then

$$p_k^{(T)} = \frac{e^{z_k/T}}{\sum_j e^{z_j/T}} \approx \frac{1 + z_k/T}{K + \sum_j z_j/T} = \frac{1 + z_k/T}{K} = \frac{1}{K} + \frac{z_k}{KT}.$$

Substituting into Theorem 11.1:

$$\frac{\partial \text{KL}}{\partial z_{s,k}} \approx \frac{1}{T} \cdot \frac{z_{s,k} - z_{t,k}}{KT} = \frac{z_{s,k} - z_{t,k}}{KT^2}.$$

Two things follow.

- The gradient shrinks like $1/T^2$. Multiplying the KL term by T^2 in (11.1) keeps its size comparable to the cross-entropy term whatever T we choose.
- At high temperature, distillation is approximately *matching the teacher’s logits*: the gradient is proportional to $z_{s,k} - z_{t,k}$, which is the gradient of $\frac{1}{2} \sum_k (z_{s,k} - z_{t,k})^2$.

Worked example: a distillation gradient

Teacher logits $(2, 1, 0.1)$, student logits $(1, 1, 1)$, $T = 4$. Then $q = \text{softmax}(0.5, 0.25, 0.025) = (0.417, 0.324, 0.259)$ and the student is uniform, $(0.333, 0.333, 0.333)$. By Theorem 11.1, the gradient is

$$\frac{1}{4} (0.333 - 0.417, 0.333 - 0.324, 0.333 - 0.259) = (-0.021, 0.002, 0.019).$$

The high-temperature approximation gives $(-0.020, 0.001, 0.019)$, already close at $T = 4$. Gradient descent raises the student’s “cat” logit and lowers “car”, copying the teacher’s ranking of the classes.

11.5 Dataset distillation: a different idea with a similar name

Dataset distillation [6] does not involve a teacher and student. It *synthesises* a very small set of artificial images (for example, ten images per class) such that a model trained only on them does as well as possible. The images are optimised directly by gradient descent. It is powerful but expensive, and the results are not real photos. Data pruning never creates images; it selects real ones.

11.6 Side-by-side comparison

	Knowledge distillation	Dataset distillation	Our project (pruning with a cheap scorer)
Goal	a small model as good as a big one	a tiny synthetic training set	cheaper training of a big model
Direction	big teacher $\rightarrow$ small student	data $\rightarrow$ synthetic data	small scorer $\rightarrow$ big model
What is passed on	soft probabilities for every image	new artificial images	a list of which images to keep
Changes the loss?	yes, adds a KL term	no	no
Changes the data?	no	replaces it	removes part of it

The honest answer to “is it related to distillation?” It is related in spirit: in both, one model helps train another. But the direction is reversed (small helps big) and what is passed on is far less (a keep-list, not probabilities). The closest name for our setting is a *proxy model* [3]. Keep this distinction clear in the proposal and the viva.

11.7 An optional bonus experiment that uses distillation

Pruning loses information: the dropped images, and the “dark knowledge” of which classes look alike. A natural stretch experiment is to train the pruned ResNet-18 with the loss (11.1), using soft targets from some teacher, and check whether this recovers the accuracy lost by pruning. If you try this, apply the rule of Chapter 10: **the teacher’s training cost and the cost of computing its soft targets count as part of the total**. A teacher that took a full training run to produce defeats the purpose unless it is reused many times. One cheap option is to use the scorer itself as the teacher, since it has already been trained.

Check yourself

1. Why is the KL term multiplied by T^2 ? (*Answer: its gradient scales like $1/T^2$; the factor keeps the two loss terms balanced.*)
2. In one sentence, how does our project differ from knowledge distillation? (*Answer: a small model selects data for a big one, passing only a keep-list, instead of a big model passing soft labels to a small one.*)

Chapter 12

Doing Experiments Properly

Deep learning results are noisy. Two runs of exactly the same code with different random seeds can differ by several tenths of a percent in test accuracy. Many differences between pruning methods are about that size. This chapter shows how to tell a real difference from luck.

12.1 Where the randomness comes from

A *random seed* fixes the random number generator. Different seeds change:

- the initial weights of the network;
- the order in which mini-batches are drawn;
- the random crops and flips of data augmentation;
- for random pruning, *which* examples are kept.

To estimate how much results vary, we repeat each setting with several seeds (at least 3) and report the average and the spread.

12.2 Mean, standard deviation and the $n - 1$

For n results $x_1, \dots, x_n$ (for example, test accuracies from n seeds):

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i, \quad s = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2}.$$

The *sample standard deviation* s measures the typical seed-to-seed variation. Why divide by $n - 1$ rather than n ?

Theorem 12.1. *If $x_1, \dots, x_n$ are independent with mean μ and variance σ^2 , then $\mathbb{E} [\sum_i (x_i - \bar{x})^2] = (n - 1) \sigma^2$. Hence s^2 (dividing by $n - 1$) is an unbiased estimate of σ^2 .*

Proof. Expand the sum: $\sum_i (x_i - \bar{x})^2 = \sum_i x_i^2 - 2\bar{x} \sum_i x_i + n\bar{x}^2 = \sum_i x_i^2 - n\bar{x}^2$, because $\sum_i x_i = n\bar{x}$. Now use $\mathbb{E}[X^2] = \text{Var}(X) + (\mathbb{E}X)^2$ for each term. For each x_i : $\mathbb{E}[x_i^2] = \sigma^2 + \mu^2$. For the mean,

$\text{Var}(\bar{x}) = \sigma^2/n$ (the averaging result of Chapter 7), so $\mathbb{E}[\bar{x}^2] = \sigma^2/n + \mu^2$. Therefore

$$\mathbb{E}\left[\sum_i (x_i - \bar{x})^2\right] = n(\sigma^2 + \mu^2) - n\left(\frac{\sigma^2}{n} + \mu^2\right) = (n-1)\sigma^2. \quad \square$$

In plain words

The data are, on average, a little closer to their own mean $\bar{x}$ than to the true mean μ , because $\bar{x}$ was computed from those same data. Dividing by $n-1$ corrects for this.

12.3 Standard error and confidence intervals

The standard deviation describes single runs. How precisely do we know the *mean*? Since $\text{Var}(\bar{x}) = \sigma^2/n$, the *standard error* of the mean is

$$\text{SE} = \frac{s}{\sqrt{n}}.$$

With few runs we use the *t*-distribution to build a 95% confidence interval:

$$\bar{x} \pm t_{0.975, n-1} \cdot \frac{s}{\sqrt{n}},$$

where $t_{0.975, n-1}$ comes from a table (or `scipy.stats.t.ppf(0.975, n-1)`); it is 4.30 for $n = 3$ runs, 2.78 for $n = 5$ and approaches 1.96 for many runs.

Worked example: three seeds

Method A (keep hard, $f = 0.5$): accuracies 93.1, 93.5, 92.9.

- Mean: $\bar{x} = 279.5/3 = 93.167$.
- Deviations: $-0.067, 0.333, -0.267$; squares sum to 0.1867; $s = \sqrt{0.1867/2} = 0.306$.
- Standard error: $0.306/\sqrt{3} = 0.176$.
- 95% confidence interval: $93.167 \pm 4.30 \times 0.176 = 93.17 \pm 0.76$, i.e. $[92.41, 93.93]$.

12.4 Is the difference between two methods real?

Compare method A above with method B (random, $f = 0.5$): 92.4, 92.9, 92.6, with mean 92.633 and $s = 0.252$. A is ahead by 0.533 points. Is that real?

Welch's *t*-test compares two means without assuming equal spreads:

$$t = \frac{\bar{x}_A - \bar{x}_B}{\sqrt{s_A^2/n_A + s_B^2/n_B}}.$$

Worked example: a difference that is not yet convincing

$\sqrt{0.306^2/3 + 0.252^2/3} = 0.229$, so $t = 0.533/0.229 = 2.33$. The effective degrees of freedom (Welch's formula) are about 3.9, and the 5% critical value is about 2.82. Since $2.33 < 2.82$, the difference is *not* statistically significant ($p \approx 0.08$).

This is typical: **with three seeds, a difference of half a percent can be luck.** What to do:

- Run more seeds for the comparisons that matter most. If the seed-to-seed standard deviation is about 0.3, five seeds per method would typically be enough to detect a 0.5-point difference (the standard error of the difference becomes $0.3\sqrt{2/5} \approx 0.19$).
- Always show error bars (mean $\pm$ standard deviation or standard error, stating which).
- Describe small differences honestly: “A was ahead by 0.5 points, within seed variation”.
- Look for consistency: if A beats B at every keep fraction and on both datasets, that pattern is more convincing than any single comparison.

12.5 A fair-comparison checklist

Within one comparison, everything except the thing being compared must be identical:

- same model, optimiser, learning-rate schedule, batch size, weight decay and augmentation;
- same number of epochs, with the equal-epochs or equal-steps choice stated (Chapter 6);
- same seeds across methods (seed 0, 1, 2 for every method), so that any shared randomness affects them equally;
- same GPU type for any timing comparison;
- test accuracy always on the full 10,000-image test set, never on training data;
- no decision made by looking at test accuracy (choose settings in advance or on a validation split).

12.6 How many runs can we afford?

Experiment	Runs	Notes
Full-data baseline	3	one per seed
Main grid: 4 fractions $\times$ 3 strategies $\times$ 3 seeds	36	cheaper than full runs
Scorers: 2 models $\times$ 3 epoch settings $\times$ 3 runs	18	short runs
Cheap-scorer grid: 2 scorers $\times$ 4 fractions $\times$ 3 seeds	24	keep-hard only
Short-schedule baseline	3–12	see Chapter 10

Because pruned runs use only a fraction of the data, the grid costs far less than 36 full runs: with fractions 0.9, 0.7, 0.5, 0.3, the 36 runs cost $9 \times (0.9 + 0.7 + 0.5 + 0.3) = 21.6$ full-run equivalents. Knowing the time of one full run on your GPU, you can turn this into hours and divide it across the four team members' free GPU quotas.

Check yourself

1. Accuracies 94.0, 94.4, 94.2: compute the mean and s . (*Answer: 94.2 and 0.2.*)
2. Why use the same seeds for all methods? (*Answer: so that differences come from the method, not from different random draws.*)

Chapter 13

Implementation in PyTorch

This chapter gives a complete, working implementation of the project, split into small files. Every file was run end to end in a smoke test (random fake data on a CPU) while preparing this guide. On Kaggle or Colab you will run them on the real CIFAR data with a GPU.

13.1 File layout

File	Purpose
data.py	datasets, augmentation, and an index-returning wrapper
models.py	ResNet-18 for CIFAR and a tiny CNN scorer
train.py	one training run: mixed precision, cosine schedule, timing, forgetting
scores.py	EL2N scores averaged over several scorer runs
prune.py	keep hard / easy / random, optional class balance and offset
run.py	the main experiment grid, logged to CSV
analysis.py	plots, scorer agreement, and the hardest images

13.2 Data

Two copies of the training set are loaded: one *with* augmentation for training and one *without* for scoring (scores must be computed on the plain image; Chapter 7). The `WithIndex` wrapper returns each image's position in the dataset, which is needed to count forgetting events per image and to train on a chosen subset.

```
# data.py -- datasets, transforms, and a wrapper that also returns each image's index
import torchvision
import torchvision.transforms as T
from torch.utils.data import Dataset

MEAN, STD = (0.4914, 0.4822, 0.4465), (0.2470, 0.2435, 0.2616)
train_tf = T.Compose([T.RandomCrop(32, padding=4), T.RandomHorizontalFlip(),
                     T.ToTensor(), T.Normalize(MEAN, STD)])
plain_tf = T.Compose([T.ToTensor(), T.Normalize(MEAN, STD)]) # no augmentation

class WithIndex(Dataset):
    """Returns (image, label, index) so we always know which image is which."""
    def __init__(self, ds):
```

```

        self.ds = ds
    def __len__(self):
        return len(self.ds)
    def __getitem__(self, i):
        x, y = self.ds[i]
        return x, y, i

def get_cifar(root="./data", name="cifar10"):
    C = torchvision.datasets.CIFAR10 if name == "cifar10" else torchvision.datasets.CIFAR100
    train_aug = C(root, train=True, download=True, transform=train_tf) # for training
    train_plain = C(root, train=True, download=True, transform=plain_tf) # for scoring
    test = C(root, train=False, download=True, transform=plain_tf)
    return train_aug, train_plain, test

```

13.3 Models

The ResNet-18 is the standard CIFAR variant: a 3×3 first convolution with no initial down-sampling (the ImageNet version would shrink 32×32 images far too early). The shortcut implements $y = x + F(x)$ from Chapter 4. It has 11,173,962 parameters.

```

# models.py -- ResNet-18 (CIFAR version) and a tiny CNN scorer
import torch.nn as nn
import torch.nn.functional as F

class BasicBlock(nn.Module):
    def __init__(self, cin, cout, stride=1):
        super().__init__()
        self.c1 = nn.Conv2d(cin, cout, 3, stride, 1, bias=False)
        self.b1 = nn.BatchNorm2d(cout)
        self.c2 = nn.Conv2d(cout, cout, 3, 1, 1, bias=False)
        self.b2 = nn.BatchNorm2d(cout)
        self.shortcut = nn.Sequential() # identity: y = x + F(x)
        if stride != 1 or cin != cout: # match shapes when they change
            self.shortcut = nn.Sequential(nn.Conv2d(cin, cout, 1, stride, bias=False),
                                          nn.BatchNorm2d(cout))

    def forward(self, x):
        out = F.relu(self.b1(self.c1(x)))
        out = self.b2(self.c2(out))
        return F.relu(out + self.shortcut(x))

class ResNet18(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        self.stem = nn.Sequential(nn.Conv2d(3, 64, 3, 1, 1, bias=False),
                                   nn.BatchNorm2d(64), nn.ReLU())

        layers, cin = [], 64
        for cout, stride in [(64, 1), (128, 2), (256, 2), (512, 2)]:
            layers += [BasicBlock(cin, cout, stride), BasicBlock(cout, cout, 1)]
            cin = cout
        self.layers = nn.Sequential(*layers)
        self.fc = nn.Linear(512, num_classes)

    def forward(self, x):
        x = self.layers(self.stem(x))
        x = F.adaptive_avg_pool2d(x, 1).flatten(1)
        return self.fc(x) # logits

def tiny_cnn(num_classes=10):

```

```
def block(cin, cout):
    return nn.Sequential(nn.Conv2d(cin, cout, 3, padding=1, bias=False),
                          nn.BatchNorm2d(cout), nn.ReLU(), nn.MaxPool2d(2))
return nn.Sequential(block(3, 32), block(32, 64), block(64, 128),
                      nn.AdaptiveAvgPool2d(1), nn.Flatten(), nn.Linear(128, num_classes))
```

13.4 Training

Key points in `train.py`:

- `Subset(ds, indices)` trains on the pruned set only; one epoch then has fewer steps, which is where the saving comes from (equal epochs).
- **Mixed precision** (`torch.autocast` with `float16` and a `GradScaler`) often gives a large speed-up (commonly close to $2\times$ on a T4) with no loss in accuracy.
- The timer is wrapped in `torch.cuda.synchronize()` (Chapter 10).
- Forgetting events are counted from the predictions already made on each mini-batch, so tracking them is essentially free. Note that correctness is measured on the augmented image at the moment it is trained on, as in Toneva et al.

```
# train.py -- one training run, with timing and optional forgetting tracking
import time
import numpy as np
import torch
import torch.nn.functional as F
from torch.utils.data import DataLoader, Subset
from data import WithIndex

NUM_WORKERS = 2

def amp(device):
    """Mixed precision (16-bit) on the GPU; plain 32-bit on the CPU."""
    return torch.autocast(device_type=device, dtype=torch.float16,
                          enabled=(device == "cuda"))

def sync(device):
    if device == "cuda":
        torch.cuda.synchronize() # GPU work is asynchronous: wait before reading the clock

@torch.no_grad()
def evaluate(model, test_ds, device="cuda", bs=512):
    model.eval()
    correct = 0
    for x, y in DataLoader(test_ds, batch_size=bs, num_workers=NUM_WORKERS):
        x, y = x.to(device), y.to(device)
        with amp(device):
            out = model(x)
            correct += (out.argmax(1) == y).sum().item()
    return correct / len(test_ds)

def train(model, train_ds, test_ds, epochs, indices=None, lr=0.1, bs=128,
          seed=0, device="cuda", track_forgetting=False):
    torch.manual_seed(seed)
    np.random.seed(seed)
    ds = WithIndex(train_ds)
```

```

if indices is not None:                                # train on the pruned subset only
    ds = Subset(ds, indices)
    loader = DataLoader(ds, batch_size=bs, shuffle=True, num_workers=NUM_WORKERS,
                        pin_memory=(device == "cuda"))
    model = model.to(device)
    opt = torch.optim.SGD(model.parameters(), lr=lr, momentum=0.9,
                          weight_decay=5e-4, nesterov=True)
    sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=epochs * len(loader))
    scaler = torch.amp.GradScaler(device, enabled=(device == "cuda"))

N = len(train_ds)
prev = np.zeros(N, dtype=bool)                        # was the example correct last time?
forgets = np.zeros(N, dtype=np.int32)                 # number of forgetting events
learned = np.zeros(N, dtype=bool)                     # ever classified correctly?

sync(device); t0 = time.time()
for ep in range(epochs):
    model.train()
    for x, y, idx in loader:
        x, y = x.to(device, non_blocking=True), y.to(device, non_blocking=True)
        with amp(device):                             # mixed precision
            out = model(x)
            loss = F.cross_entropy(out, y)
            opt.zero_grad(set_to_none=True)
            scaler.scale(loss).backward()
            scaler.step(opt)
            scaler.update()
            sched.step()
        if track_forgetting:
            correct = (out.argmax(1) == y).cpu().numpy()
            idx = idx.numpy()
            forgets[idx] += prev[idx] & ~correct      # a 1 -> 0 transition
            prev[idx] = correct
            learned[idx] |= correct
    sync(device); train_time = time.time() - t0

acc = evaluate(model, test_ds, device)
extra = {}
if track_forgetting:
    f = forgets.astype(float)
    f[~learned] = f.max() + 1                          # never learned = hardest of all
    extra["forgetting"] = f
return model, acc, train_time, extra

```

13.5 Scores

`el2n_scores` computes $\|p - y\|_2$ for every training image, in order (`shuffle=False`), in evaluation mode. `compute_el2n` trains several scorers and averages their scores, and returns the *total* time spent (training plus scoring), which is the *S* of Chapter 10.

A design choice to state in the report

Here each scorer is trained for e epochs with its own complete (shortened) learning-rate schedule. Paul et al. instead take a snapshot after e epochs of a run whose schedule was set for the full length. Both are reasonable; ours is simpler and slightly favours early

scoring. Say which one you used.

```
# scores.py -- EL2N scores, averaged over several independently trained scorers
import time
import numpy as np
import torch
import torch.nn.functional as F
from torch.utils.data import DataLoader
from train import train, sync, amp, NUM_WORKERS

@torch.no_grad()
def el2n_scores(model, train_plain, num_classes, device="cuda", bs=512):
    model.eval() # use stored BatchNorm statistics
    out = []
    loader = DataLoader(train_plain, batch_size=bs, shuffle=False, # keep order!
                        num_workers=NUM_WORKERS)
    for x, y in loader:
        x, y = x.to(device), y.to(device)
        with amp(device):
            logits = model(x)
            p = F.softmax(logits.float(), dim=1)
            err = p - F.one_hot(y, num_classes).float() # the error vector p - y
            out.append(err.norm(dim=1).cpu()) # its length = EL2N
    return torch.cat(out).numpy()

def compute_el2n(make_model, train_aug, train_plain, test, num_classes,
                 epochs, runs, device="cuda"):
    """Train 'runs' scorers for 'epochs' epochs each; return mean EL2N and total cost."""
    all_scores, total_time = [], 0.0
    for r in range(runs):
        model, acc, t_train, _ = train(make_model(), train_aug, test, epochs,
                                       seed=1000 + r, device=device)

        sync(device); t0 = time.time()
        s = el2n_scores(model, train_plain, num_classes, device)
        sync(device); total_time += t_train + (time.time() - t0)
        all_scores.append(s)
        print(f"scorer run {r}: test acc {acc:.3f}, {t_train:.0f}s")
    return np.mean(all_scores, axis=0), total_time
```

13.6 Pruning

`select` returns the indices to keep. With `class_balanced=True` it keeps the fraction f of every class separately; because of rounding per class, the total can differ from fN by a few examples. `offset` skips the very hardest examples (possible label noise) before keeping the next-hardest.

```
# prune.py -- choose which examples to keep
import numpy as np

def select(scores, f, strategy, labels=None, seed=0, class_balanced=False, offset=0.0):
    """Return sorted indices of the examples to keep.
    scores: array of length N, larger = harder.  f: keep fraction in (0, 1].
    strategy: 'hard', 'easy' or 'random'.  offset: fraction of the very hardest to skip."""
    rng = np.random.default_rng(seed)

    def pick(idx):
        n = int(round(f * len(idx)))
```

```

    if strategy == "random":
        return rng.choice(idx, n, replace=False)
    order = idx[np.argsort(-scores[idx], kind="stable")]  # hardest first
    if strategy == "hard":
        skip = int(round(offset * len(idx)))
        return order[skip:skip + n]
    if strategy == "easy":
        return order[::-1][:n]
    raise ValueError(strategy)

if class_balanced:  # keep the same fraction of every class
    labels = np.asarray(labels)
    kept = [pick(np.where(labels == c)[0]) for c in np.unique(labels)]
    return np.sort(np.concatenate(kept))
return np.sort(pick(np.arange(len(scores))))

```

13.7 The experiment grid

`run.py` scores once, saves the scores to a `.npy` file, and then runs every combination of keep fraction, strategy and seed, appending one line per run to `results.csv`. If a Kaggle session dies, re-running the script continues from the saved scores. To make the grid resumable too, skip any (fraction, strategy, seed) that already appears in the CSV.

```

# run.py -- the main pruning grid, logging every run to a CSV file
import csv, os
import numpy as np
from data import get_cifar
from models import ResNet18, tiny_cnn
from scores import compute_el2n
from prune import select
from train import train

def log_row(path, **row):
    new = not os.path.exists(path)
    with open(path, "a", newline="") as fh:
        w = csv.DictWriter(fh, fieldnames=list(row))
        if new:
            w.writeheader()
        w.writerow(row)

if __name__ == "__main__":
    DEVICE, EPOCHS, K = "cuda", 60, 10
    train_aug, train_plain, test = get_cifar(name="cifar10")
    labels = np.array(train_plain.targets)

    # 1) scores from a cheap scorer (saved once, reused by everyone)
    if not os.path.exists("el2n_tiny_e5.npy"):
        s, cost = compute_el2n(lambda: tiny_cnn(K), train_aug, train_plain, test, K,
                               epochs=5, runs=3, device=DEVICE)
        np.save("el2n_tiny_e5.npy", s)
        log_row("scoring_costs.csv", scorer="tiny", epochs=5, runs=3,
                seconds=round(cost, 1))
    scores = np.load("el2n_tiny_e5.npy")

    # 2) the grid: keep fraction x strategy x seed
    for f in [0.9, 0.7, 0.5, 0.3]:

```

```

for strategy in ["hard", "easy", "random"]:
    for seed in [0, 1, 2]:
        idx = select(scores, f, strategy, labels=labels, seed=seed)
        _, acc, t, _ = train(ResNet18(K), train_aug, test, EPOCHS,
                             indices=idx, seed=seed, device=DEVICE)
        log_row("results.csv", model="resnet18", scorer="tiny_e5",
                strategy=strategy, fraction=f, seed=seed, n_kept=len(idx),
                test_acc=round(acc, 4), train_seconds=round(t, 1))

```

13.8 Analysis

plot_grid draws the “accuracy versus data kept” plot with error bars. agreement prints Spearman’s ρ and the kept-set overlaps next to their random baselines (Chapter 9). show_hardest saves a grid of the highest-scored images with their labels, a quick check for mislabelled images and a good slide.

```

# analysis.py -- plots, scorer agreement, and a look at the hardest images
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import spearmanr

def plot_grid(csv="results.csv", out="acc_vs_fraction.png"):
    df = pd.read_csv(csv)
    g = (df.groupby(["strategy", "fraction"])["test_acc"]
         .agg(["mean", "std", "count"]).reset_index())
    for strategy, sub in g.groupby("strategy"):
        plt.errorbar(sub["fraction"] * 100, sub["mean"] * 100, yerr=sub["std"] * 100,
                     marker="o", capsize=3, label=strategy)
    plt.xlabel("% of training data kept"); plt.ylabel("test accuracy (%)")
    plt.legend(); plt.tight_layout(); plt.savefig(out, dpi=200); plt.close()
    return g

def overlap(a, b, f):
    """Fraction of the n hardest under score a that are also among the n hardest under b."""
    n = int(f * len(a))
    A = set(np.argsort(-a)[:n]); B = set(np.argsort(-b)[:n])
    return len(A & B) / n

def agreement(a, b, fractions=(0.9, 0.7, 0.5, 0.3)):
    rho = spearmanr(a, b).correlation
    print(f"Spearman rho = {rho:.3f}")
    for f in fractions:
        # compare with the random-overlap baseline f
        print(f"keep {f:.0%}: overlap {overlap(a, b, f):.3f} (random: {f:.2f})")

def show_hardest(scores, k=20, out="hardest.png", root="./data"):
    import torch, torchvision, torchvision.transforms as T, torchvision.utils as vu
    raw = torchvision.datasets.CIFAR10(root, train=True, download=True)
    idx = np.argsort(-scores)[:k]
    imgs = torch.stack([T.ToTensor()(raw[i][0]) for i in idx])
    vu.save_image(imgs, out, nrow=10)
    print("labels:", [raw.classes[raw.targets[i]] for i in idx])

```

13.9 Working on Kaggle or Colab

- **Turn on the GPU** (Kaggle: *Settings* → *Accelerator*; Colab: *Runtime* → *Change runtime type*). Check with `torch.cuda.get_device_name()` and record the name.
- **Save to persistent storage.** On Kaggle, write to `/kaggle/working/` and save a version of the notebook to keep outputs; on Colab, mount Google Drive. Sessions end without warning.
- **Share score files.** The team's most valuable files are the `.npy` scores. Upload them to a shared Kaggle dataset or Drive folder so nobody recomputes them.
- **Use a quick test first.** Run the grid with 2 epochs and 1 seed to check that everything works before spending hours.
- **Free GPU quotas are limited** (Kaggle gives a weekly GPU-hour allowance per account; check the current number). Four accounts give four times the budget; split the grid by seed or by keep fraction.
- `num_workers=2` is a good default on both platforms; more workers can crash on Kaggle's limited CPU memory.

Chapter 14

Project Plan and Viva Preparation

This final chapter turns everything into a plan for a team of four and collects the questions you are most likely to be asked.

14.1 Timeline

When	What
By Oct 1	4-slide proposal submitted.
Week of Oct 1	Shared training script working; full-data ResNet-18 baseline (3 seeds) with timings; everyone reads Chapters 1–8.
Oct 5–12	EL2N scores from ResNet-18 (reference) and tiny CNN at 1, 5, 20 epochs; forgetting scores; sanity check of the hardest images.
Oct 12–24	Main grid on CIFAR-10: hard / easy / random at 90, 70, 50, 30% with 3 seeds; accuracy-vs-data-kept plot.
Oct 26	Mid-project presentation: baseline, scores, main plot, first observations.
Oct 27–Nov 10	Cheap-scorer grid; scorer agreement (Spearman, overlap); scoring-epoch study; cost table with the short-schedule baseline.
Nov 10–18	Key experiments on CIFAR-100; optional class-balanced or offset variants; optional distillation bonus.
Nov 18–24	Final plots, cost–accuracy scatter plot, report writing.
Nov 24 / 26	Report submitted / final presentation and viva.

14.2 Team roles

Member	Owns	Chapters to master
A: pipeline	<code>data.py</code> , <code>models.py</code> , <code>train.py</code> ; baselines; timing; making sure everyone runs identical code	3, 4, 10, 13
B: scoring	<code>scores.py</code> ; EL2N and forgetting; tiny-CNN scorer; score files for 1/5/20 epochs	4, 7, 9
C: pruning grid	<code>prune.py</code> , <code>run.py</code> ; the main grid; CIFAR-100	6, 8, 12
D: analysis & story	<code>analysis.py</code> ; agreement and cost study; plots; slides and report structure	9, 10, 11, 12

A must finish first because everyone depends on the training script. Everyone runs experiments on their own GPU account, including for parts they do not own, and everyone should be able to answer questions on any part.

14.3 Likely viva questions, with short answers

1. **What is data pruning?** Training on a chosen subset of the training set to reduce cost, ideally with little loss in accuracy.
2. **What is EL2N?** The length of the error vector $\mathbf{p} - \mathbf{y}$ after a few epochs of training, averaged over a few runs. Large means the model is far from confidently correct.
3. **Why is EL2N a reasonable proxy for the gradient norm?** The weight gradient equals $J^\top(\mathbf{p} - \mathbf{y})$, so its norm is at most $\|J\| \|\mathbf{p} - \mathbf{y}\|$; for the last layer it equals EL2N times the feature norm exactly.
4. **What values can EL2N take?** Between $1 - p_y$ and $\sqrt{2}(1 - p_y)$, so between 0 and $\sqrt{2}$.
5. **Derive the gradient of cross-entropy with respect to the logits.** $\partial\ell/\partial z_j = -\frac{1}{p_y} p_y(\delta_{yj} - p_j) = p_j - y_j$, using the softmax derivative.
6. **Why is random pruning a strong baseline?** The loss on a random subset is an unbiased estimate of the full-data loss (indicator variables plus linearity of expectation), so it optimises the right objective, just with less data.
7. **When should you keep hard and when easy examples?** Keep hard when you keep a lot of data; keep easy when you keep very little. With little data the model needs coarse structure, which easy examples give robustly.
8. **Why might a cheap scorer fail?** Capacity mismatch and a slightly different decision boundary. In the 1D toy, a scorer off by δ fails when $f < 2\delta$: aggressive pruning needs an accurate scorer.

9. **How do you account for compute?** Total = $S + fF$ with equal epochs. It saves compute only if $f < 1 - S/F$. We also compare against training on all data for a shorter schedule of equal cost.
10. **Equal epochs or equal steps?** Equal epochs, because it is the setting in which pruning can save compute; equal steps costs at least as much as full training.
11. **Is this knowledge distillation?** No. Distillation passes a big teacher's soft probabilities to a small student. Here a small model passes only a keep-list to a big model; the closest name is a proxy model.
12. **How do you know a difference is real?** Multiple seeds, error bars, a Welch t -test; with 3 seeds a 0.5-point difference can easily be luck.
13. **Why average EL2N over several runs?** It reduces the variance of the score by a factor k (standard deviation by $\sqrt{k}$), at k times the cost.
14. **What do the highest-scored images look like?** Typically ambiguous, unusual or mislabelled images. Showing them is a sanity check.
15. **How do you compare two scorers without training?** Spearman's ρ on the whole ranking and the overlap of kept sets, compared with the random-overlap baseline f .
16. **Why ResNet-18 and not ResNet-50?** Free GPUs (T4/P100) make ResNet-50 runs several times slower; ResNet-18 lets us run all seeds and fractions.
17. **What does the forgetting score measure?** The number of times an example goes from correctly to incorrectly classified during training; never-learned examples get the maximum score.
18. **What could go wrong with class balance?** Keeping "the hardest globally" can over-represent hard classes (cat, dog) and nearly remove easy ones; class-balanced pruning fixes this.
19. **What is your main contribution?** Measuring whether pruning with cheap and early scores saves compute overall, with the scoring cost counted, rather than only reporting accuracy versus data kept.
20. **What would you do next?** Dynamic pruning during training, better cheap scorers, or reusing a pruned set across many runs, which divides the scoring cost by the number of reuses.

14.4 A final checklist

- ☐ Random pruning at every keep fraction, with the same seeds as the other strategies.
- ☐ At least 3 seeds everywhere; error bars on every plot.
- ☐ Equal-epochs setting stated clearly.
- ☐ Scoring time measured and included in the cost table.
- ☐ Short-schedule full-data baseline in the cost table.
- ☐ GPU type recorded with every timing.

- ☐ Test accuracy on the full test set only; no tuning on test data.
- ☐ Images of the hardest and easiest examples.
- ☐ Every team member can derive $p - y$ and explain the break-even condition.

Appendix A

Mathematics Refresher

This appendix collects the mathematical tools used in the guide, with short proofs.

A.1 Vectors, dot products and norms

A vector $\mathbf{a} = (a_1, \dots, a_d)$ is a list of d numbers.

- **Dot product:** $\mathbf{a} \cdot \mathbf{b} = \sum_{j=1}^d a_j b_j$.
- **Length (Euclidean or L_2 norm):** $\|\mathbf{a}\|_2 = \sqrt{\mathbf{a} \cdot \mathbf{a}} = \sqrt{\sum_j a_j^2}$. This is the “L2” in EL2N.
- **Distance between $\mathbf{a}$ and $\mathbf{b}$:** $\|\mathbf{a} - \mathbf{b}\|_2$.

Theorem A.1 (Cauchy–Schwarz inequality). *For any vectors $\mathbf{a}, \mathbf{b}$: $|\mathbf{a} \cdot \mathbf{b}| \leq \|\mathbf{a}\| \|\mathbf{b}\|$, with equality exactly when one is a multiple of the other.*

Proof. If $\mathbf{b} = \mathbf{0}$ both sides are 0. Otherwise, for every real number λ ,

$$0 \leq \|\mathbf{a} - \lambda \mathbf{b}\|^2 = \|\mathbf{a}\|^2 - 2\lambda \mathbf{a} \cdot \mathbf{b} + \lambda^2 \|\mathbf{b}\|^2.$$

Choose $\lambda = (\mathbf{a} \cdot \mathbf{b}) / \|\mathbf{b}\|^2$ (the value that minimises the right-hand side). Substituting gives $0 \leq \|\mathbf{a}\|^2 - (\mathbf{a} \cdot \mathbf{b})^2 / \|\mathbf{b}\|^2$, which rearranges to $(\mathbf{a} \cdot \mathbf{b})^2 \leq \|\mathbf{a}\|^2 \|\mathbf{b}\|^2$. Equality needs $\mathbf{a} - \lambda \mathbf{b} = \mathbf{0}$. $\square$

A.2 Matrices

A matrix A with m rows and n columns maps a vector $\mathbf{v} \in \mathbb{R}^n$ to $A\mathbf{v} \in \mathbb{R}^m$, where $(A\mathbf{v})_i = \sum_j A_{ij} v_j$. The *transpose* $A^\top$ swaps rows and columns.

- **Operator norm:** $\|A\|_{\text{op}} = \max_{\mathbf{v} \neq \mathbf{0}} \|A\mathbf{v}\| / \|\mathbf{v}\|$, the largest factor by which A can stretch a vector. By definition, $\|A\mathbf{v}\| \leq \|A\|_{\text{op}} \|\mathbf{v}\|$ for every $\mathbf{v}$. A matrix and its transpose have the same operator norm.
- **Frobenius norm:** $\|A\|_F = \sqrt{\sum_{i,j} A_{ij}^2}$, the length of the matrix treated as one long vector.

A.3 Exponentials and logarithms

We use the natural logarithm $\ln$ (written $\log$ in the main text). Useful facts: $e^{a+b} = e^a e^b$; $\ln(ab) = \ln a + \ln b$; $\ln$ is increasing, so maximising a positive quantity is the same as maximising its logarithm; $\frac{d}{dx} e^x = e^x$ and $\frac{d}{dx} \ln x = 1/x$; and for small u , $e^u \approx 1 + u$ (first-order Taylor approximation).

A.4 Derivative rules

- **Chain rule:** $\frac{d}{dx} g(u(x)) = g'(u(x)) u'(x)$.
- **Quotient rule:** $\left(\frac{f}{g}\right)' = \frac{f'g - fg'}{g^2}$.
- For many variables, the chain rule through intermediate variables $u_1, \dots, u_K$ reads $\frac{\partial \ell}{\partial w} = \sum_k \frac{\partial \ell}{\partial u_k} \frac{\partial u_k}{\partial w}$.

A.5 Probability and expectation

For a random quantity X that takes value x with probability $\Pr(X = x)$:

- **Expectation** (average value): $\mathbb{E}[X] = \sum_x x \Pr(X = x)$.
- **Linearity:** $\mathbb{E}[aX + bY] = a \mathbb{E}[X] + b \mathbb{E}[Y]$ for any constants a, b and any X, Y , even dependent ones.
- **Indicator variables:** if $I = 1$ when an event happens and 0 otherwise, then $\mathbb{E}[I] = \Pr(\text{event})$.
- **Variance:** $\text{Var}(X) = \mathbb{E}[(X - \mathbb{E}X)^2] = \mathbb{E}[X^2] - (\mathbb{E}X)^2$.
- **Scaling:** $\text{Var}(aX) = a^2 \text{Var}(X)$.
- **Independence:** if X and Y are independent, $\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y)$.

Proposition A.2 (Tail formula for the expectation). *For a non-negative random variable G , $\mathbb{E}[G] = \int_0^\infty \Pr(G > t) dt$.*

Proof. For any number $g \geq 0$, $g = \int_0^g 1 dt = \int_0^\infty \mathbf{1}[g > t] dt$. Apply this to G , take expectations of both sides, and swap the expectation and the integral (allowed because everything is non-negative). Then $\mathbb{E} \mathbf{1}[G > t] = \Pr(G > t)$. $\square$

A.6 Two sums

$$\sum_{k=1}^N k = \frac{N(N+1)}{2}, \quad \sum_{k=1}^N k^2 = \frac{N(N+1)(2N+1)}{6}.$$

Proof. First: write the sum forwards and backwards and add them; each of the N columns sums to $N+1$, so twice the sum is $N(N+1)$. Second, by induction: it holds for $N = 1$

$(1 = 1 \cdot 2 \cdot 3/6)$. If it holds for N , then

$$\begin{aligned}\frac{N(N+1)(2N+1)}{6} + (N+1)^2 &= \frac{(N+1)(N(2N+1) + 6(N+1))}{6} \\ &= \frac{(N+1)(N+2)(2N+3)}{6},\end{aligned}$$

since $2N^2 + 7N + 6 = (N+2)(2N+3)$. This is the formula for $N+1$. □

Appendix B

Glossary

Augmentation

Random crops and flips applied to training images each epoch to reduce overfitting.

Batch normalisation

A layer that rescales each channel to zero mean and unit variance over a mini-batch.

Break-even keep fraction

The keep fraction $1 - S/F$ below which pruning saves compute (Chapter 10).

Cross-entropy

The classification loss $-\log p_y$.

Epoch

One full pass over the training set.

EL2N Error L2-Norm, $\|p - y\|_2$ after a few epochs, averaged over runs.

Equal epochs / equal steps

Two ways to train on a pruned set: same number of passes (cheaper) or same number of gradient steps (same cost).

Forgetting event

An example going from correctly to incorrectly classified between two consecutive presentations.

GraNd

Gradient Normed score: the norm of the loss gradient with respect to the weights.

Keep fraction f

The fraction of training examples kept after pruning.

Knowledge distillation

Training a student model to match a teacher model's soft predictions.

Logits

The raw outputs of a network, one per class, before the softmax.

Mixed precision

Doing most computation in 16-bit numbers to speed up training.

One-hot vector

A vector of zeros with a single 1 at the true class.

Proxy model

A cheaper model used in place of the target model, here to score data.

ResNet

A network built from residual blocks $y = x + F(x)$.

Scorer

The model whose predictions are used to compute pruning scores.

Seed The starting value of the random number generator; different seeds give different runs.

SGD Stochastic gradient descent: gradient steps computed on random mini-batches.

Softmax

$p_k = e^{z_k} / \sum_j e^{z_j}$, turning logits into probabilities.

Spearman's ρ / Kendall's τ

Measures of agreement between two rankings.

Standard error

$s/\sqrt{n}$, the uncertainty of a mean of n runs.

Temperature

A number T dividing the logits before softmax; larger T gives softer probabilities.

Test set

Held-out data used only to measure final performance.

References

- [1] M. Paul, S. Ganguli and G. K. Dziugaite. *Deep Learning on a Data Diet: Finding Important Examples Early in Training*. NeurIPS 2021.
- [2] B. Sorscher, R. Geirhos, S. Shekhar, S. Ganguli and A. S. Morcos. *Beyond Neural Scaling Laws: Beating Power Law Scaling via Data Pruning*. NeurIPS 2022.
- [3] C. Coleman, C. Yeh, S. Mussmann, B. Mirzasoleiman, P. Bailis, P. Liang, J. Leskovec and M. Zaharia. *Selection via Proxy: Efficient Data Selection for Deep Learning*. ICLR 2020.
- [4] M. Toneva, A. Sordoni, R. Tachet des Combes, A. Trischler, Y. Bengio and G. J. Gordon. *An Empirical Study of Example Forgetting during Deep Neural Network Learning*. ICLR 2019.
- [5] G. Hinton, O. Vinyals and J. Dean. *Distilling the Knowledge in a Neural Network*. NIPS Deep Learning Workshop, 2015.
- [6] T. Wang, J.-Y. Zhu, A. Torralba and A. A. Efros. *Dataset Distillation*. arXiv:1811.10959, 2018.
- [7] D. Arpit et al. *A Closer Look at Memorization in Deep Networks*. ICML 2017.
- [8] C. Zhang, S. Bengio, M. Hardt, B. Recht and O. Vinyals. *Understanding Deep Learning Requires Rethinking Generalization*. ICLR 2017.
- [9] V. Feldman. *Does Learning Require Memorization? A Short Tale about a Long Tail*. STOC 2020.
- [10] K. He, X. Zhang, S. Ren and J. Sun. *Deep Residual Learning for Image Recognition*. CVPR 2016.
- [11] A. Krizhevsky. *Learning Multiple Layers of Features from Tiny Images*. Technical report, University of Toronto, 2009.